离散化
Graph

离散化

```cpp
template<class T>
struct discrete
{
    std::vector<T>a;
    int len;
    discrete(std::vector<T> arr,int st = 0)
    {
        std::sort(arr.begin() + st,arr.end());
        arr.erase(std::unique(arr.begin() + st,arr.end()),arr.end());
        a.assign(arr.begin() + st, arr.end());
        len = a.size();
    }
    int size(){return len;}
    //第几小
    int operator[](T x){
        int pos = std::lower_bound(a.begin(),a.end(),x) - a.begin();
        return pos + 1;
    }
};
```

Graph

```cpp
struct Graph
{
    struct edge
    {
        int nx;
        int to;
        int w;
    };
    std::vector<edge>e;
    std::vector<int>heads;
    Graph(int n = 0,int m = 0) : heads(n + 1,-1)
    {
        e.reserve(m);
    }
    void add(int u,int v,int w = 0)
    {
        e.push_back({heads[u],v,w});
        heads[u] = e.size() - 1;
    }
};
//example
void solve()
```

```
{
    int n,m;
    std::cin >> n >> m;
    Graph g(n,m);
    for(int i = 1;i <= m;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g.add(u,v);
    }
    for(int u = 1;u <= n;u++)
    {
        for(int i = g.heads[u];~i;i = g.e[i].nx)
        {
            int v = g.e[i].to;
        }
    }
}
```

动态维护中位数
并查集
树状数组
二维树状数组
线段树（区间加，区间查询）
线段树合并
树链剖分
笛卡尔树
倍增求LCA
主席树
主席树求静态区间第k小
分块
二进制分块
珂朵莉树
数位dp

数据结构

动态维护中位数

```
struct Median
{
    std::priority_queue<int> L;
    std::priority_queue<int,std::vector<int>,std::greater<int>> R;
    int sl = 0, sr = 0;
    void add(int x)
    {
        if(L.empty() || x <= L.top())
        {
            L.push(x);
            sl += x;
        }
        else
        {
            R.push(x);
            sr += x;
        }

        while(L.size() > R.size() + 1)
        {
            int x = L.top();
            L.pop();
            sl -= x;

            R.push(x);
            sr += x;
        }

        while(L.size() < R.size())
```

```
        {
            int x = R.top();
            R.pop();
            sr -= x;

            L.push(x);
            sl += x;
        }
    }
    int query()
    {
        if(L.empty())return 0;
        int mid = L.top();
        return mid * L.size() - sl + sr - mid * R.size();
    }
};
```

并查集

```
struct dsu
{
    int n;
    std::vector<int>fa,siz;
    dsu(int n): n(n),fa(n + 1),siz(n + 1,1)
    {
        for(int i = 1;i <= n;i++)
            fa[i] = i;
    }
    int find(int x)
    {
        if(x == fa[x])return x;
        return fa[x] = find(fa[x]);
    }
    bool merge(int a,int b)
    {
        a = find(a);
        b = find(b);
        if(a == b)return false;
        if(siz[a] < siz[b])std::swap(a,b);
        siz[a] += siz[b];
        fa[b] = a;
        return true;
    }
};
```

树状数组

```
struct finwick
{
```

```
    int n;
    std::vector<int>t;
    finwick(int n): n(n),t(n + 1){}
    int lowbit(int x)
    {
        return x & (-x);
    }
    void add(int x,int k)
    {
        for(int i = x;i <= n;i += lowbit(i))
            t[i] += k;
    }
    int query(int l,int r)
    {
        if(l > r)return 0;
        int ans = 0;
        for(int i = r;i >= 1;i -= lowbit(i))
            ans += t[i];
        for(int i = l - 1;i >= 1;i -= lowbit(i))
            ans -= t[i];
        return ans;
    }
};
```

二维树状数组

```
struct finwick2
{
    int n,m;
    std::vector<std::vector<int>>t;
    finwick2(int n,int m): n(n),m(m),t(n + 1,std::vector<int>(m + 1)){}
    int lowbit(int x)
    {
        return x & -x;
    }
    void add(int x,int y,int k)
    {
        for(int i = x;i <= n;i += lowbit(i))
            for(int j = y;j <= m;j += lowbit(j))
                t[i][j] += k;
    }
    int query(int x,int y)
    {
        int res = 0;
        for(int i = x;i >= 1;i -= lowbit(i))
            for(int j = y;j >= 1;j -= lowbit(j))
                res += t[i][j];
        return res;
    }
    int query(int r1,int c1,int r2,int c2)
    {
        if(r1 > r2 || c1 > c2)return 0;
```

```
        return query(r2,c2) - query(r2,c1 - 1) - query(r1 - 1,c2) + query(r1 - 1,c1 - 1);
    }
};
```

线段树（区间加，区间查询）

```
struct seg
{
    struct node
    {
        int l;
        int r;
        int sum;
        int lz;
    };
    std::vector<node>t;
    void push_up(int p)
    {
        t[p].sum = (t[p << 1].sum + t[p << 1 | 1].sum);
    }
    void push_down(int p)
    {
        if(t[p].lz)
        {
            t[p << 1].sum += (t[p << 1].r - t[p << 1].l + 1) * t[p].lz;
            t[p << 1].lz += t[p].lz;
            t[p << 1 | 1].sum += (t[p << 1 | 1].r - t[p << 1 | 1].l + 1) * t[p].lz;
            t[p << 1 | 1].lz += t[p].lz;
            t[p].lz = 0;
        }
    }
    void build(int p,int l,int r)
    {
        t[p] = {l,r,0,0};
        if(l == r)
            return;
        int mid = (l + r) >> 1;
        build(p << 1,l,mid);
        build(p << 1 | 1,mid + 1,r);
        push_up(p);
    }
    seg(int n)
    {
        t.resize((n << 2) + 7);
        build(1,1,n);
    }
    void update(int p,int nl,int nr,int k)
    {
        if(nl > nr)return;
```

```
        if(nl <= t[p].l && t[p].r <= nr)
        {
            t[p].sum += (t[p].r - t[p].l + 1) * k;
            t[p].lz += k;
            return;
        }
        push_down(p);
        int mid = (t[p].l + t[p].r) >> 1;
        if(nl <= mid)update(p << 1,nl,nr,k);
        if(nr > mid)update(p << 1 | 1,nl,nr,k);
        push_up(p);
    }
    int query(int p,int nl,int nr)
    {
        if(nl > nr)return 0;
        if(nl <= t[p].l && t[p].r <= nr)
            return t[p].sum;
        push_down(p);
        int mid = (t[p].l + t[p].r) >> 1,res = 0;
        if(nl <= mid)res += query(p << 1,nl,nr);
        if(nr > mid)res += query(p << 1 | 1,nl,nr);
        return res;
    }
};

```

线段树合并

```
struct node//维护1 ~ up 最大数量及其编号
{
    int l;
    int r;
    int mxsum;
    int mxpos;
}t[N << 6];
int rt[N],tot = 0,up = 1e5;
void push_up(int p)
{
    int ls = t[p].l,rs = t[p].r;
    if(t[ls].mxsum >= t[rs].mxsum)
    {
        t[p].mxpos = t[ls].mxpos;
        t[p].mxsum = t[ls].mxsum;
    }
    else
    {
        t[p].mxpos = t[rs].mxpos;
        t[p].mxsum = t[rs].mxsum;
    }
}
```

```
int add(int id,int l,int r,int pos,int k)
{
    if(!id)id = ++tot;
    if(l == r)
    {
        t[id].mxsum += k;
        t[id].mxpos = pos;
        return id;
    }
    int mid = (l + r) >> 1;
    if(pos <= mid)t[id].l = add(t[id].l,l,mid,pos,k);
    else t[id].r = add(t[id].r,mid + 1,r,pos,k);
    push_up(id);
    return id;
}
int merge(int a,int b,int l,int r)
{
    if(!a)return b;
    if(!b)return a;
    if(l == r)
    {
        t[a].mxsum += t[b].mxsum;
        return a;
    }
    int mid = (l + r) >> 1;
    t[a].l = merge(t[a].l,t[b].l,l,mid);
    t[a].r = merge(t[a].r,t[b].r,mid + 1,r);
    push_up(a);
    return a;
}
```

树链剖分

```
const int N = 2e5+7;
struct node
{
    int l;
    int r;
    int w;
    int lz = 0;
}t[N << 2];
int a[N],wt[N];
int n, m, root, P;
int dep[N], son[N], id[N], fa[N], top[N], siz[N], tot = 0;
struct edge
{
    int nx;
    int to;
}e[N << 1];
int heads[N], cntqq = 0;
void add(int u, int v)
{
```

```
    e[++cntqq].nx = heads[u];
    e[cntqq].to = v;
    heads[u] = cntqq;
    return;
}
void push_up(int p)
{
    t[p].w = (t[p << 1].w + t[p << 1 | 1].w)%P;
    return;
}
void push_down(int p)
{
    t[p << 1].lz = (t[p << 1].lz + t[p].lz);
    t[p << 1 | 1].lz = (t[p << 1 | 1].lz + t[p].lz);
    int l1 = t[p << 1].r - t[p << 1].l + 1;
    int l2 = t[p << 1|1].r - t[p << 1|1].l + 1;
    t[p << 1].w = (t[p << 1].w + l1 * t[p].lz) % P;
    t[p << 1 | 1].w = (t[p << 1 | 1].w + l2 * t[p].lz) % P;
    t[p].lz = 0;
    return;
}
inline void build(int p, int l, int r)
{
    t[p].l = l, t[p].r = r;
    if (l == r)
    {
        t[p].w = wt[l] % P;
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    push_up(p);
}
inline void update(int p, int nl, int nr, int ww)
{

    if (nl <= t[p].l && t[p].r <= nr)
    {
        t[p].lz = (t[p].lz + ww);
        t[p].w = (t[p].w + ww * (t[p].r - t[p].l + 1))%P;
        return;
    }
    push_down(p);
    int mid = (t[p].l + t[p].r) >> 1;
    if (nl <= mid)update(p << 1, nl, nr, ww);
    if (nr > mid)update(p << 1 | 1, nl, nr, ww);
    push_up(p);
}
inline int query(int p, int nl, int nr)
{

    if (nl <= t[p].l && t[p].r <= nr)
```

```
    {
        return t[p].w % P;
    }
    push_down(p);
    int re = 0;
    int mid = (t[p].l + t[p].r) >> 1;
    if (nl <= mid)re = (re + query(p << 1, nl, nr)) % P;
    if(nr > mid)re = (re + query(p << 1 | 1, nl, nr)) % P;
    return re;
}
inline int qRange(int x, int y)
{
    int ans = 0;
    while (top[x] != top[y])
    {
        if (dep[top[x]] < dep[top[y]])swap(x, y);
        ans = (ans + query(1, id[top[x]], id[x])) % P;
        x = fa[top[x]];
    }
    if (dep[x] > dep[y])swap(x, y);
    ans = (ans + query(1, id[x], id[y])) % P;
    return ans;
}
inline void upRange(int x, int y, int k)
{
    k = k % P;
    while (top[x] != top[y])
    {
        if (dep[top[x]] < dep[top[y]])swap(x, y);
        update(1, id[top[x]], id[x], k);
        x = fa[top[x]];
    }
    if (dep[x] > dep[y])swap(x, y);
    update(1, id[x], id[y], k);
}
inline int qSon(int x)
{
    return query(1, id[x], id[x] + siz[x] - 1);
}
inline void upSon(int x, int k)
{
    update(1, id[x], id[x] + siz[x] - 1, k);
}


inline void dfs1(int p, int f, int deep)//深度，父亲，子树大小，重儿子编号
{
    dep[p] = deep;
    fa[p] = f;
    siz[p] = 1;
    int zson = -1;
    for (int i = heads[p]; i; i = e[i].nx)
    {
```

```
        int v = e[i].to;
        if (v == f)continue;
        dfs1(v, p, deep + 1);
        siz[p] += siz[v];
        if (siz[v] > zson)son[p] = v, zson = siz[v];
    }
}
inline void dfs2(int p, int topf)//top,dfs序
{
    id[p] = ++tot;
    wt[tot] = a[p];
    top[p] = topf;
    if (!son[p])return;
    dfs2(son[p], topf);
    for (int i = heads[p]; i; i = e[i].nx)
    {
        int v = e[i].to;
        if (v == fa[p] || v == son[p])continue;
        dfs2(v, v);
    }
}
inline void solve()
{
    cin >> n >> m >> root >> P;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    for (int i = 1; i <= n - 1; i++)
    {
        int u, v;
        cin >> u >> v;
        add(u, v); add(v, u);
    }
    dfs1(root, 0, 1);
    dfs2(root, root);
    build(1, 1, n);
    while (m--)
    {
        int k, x, y, z;
        cin >> k;
        switch (k)
        {
        case 1:
            cin >> x >> y >> z;
            upRange(x, y, z);
            break;
        case 2:
            cin >> x >> y;
            cout << qRange(x, y)<<endl;
            break;
        case 3:
            cin >> x >> z;
            upSon(x, z);
            break;
```

```
        case 4:
            cin >> x;
            cout << qSon(x) << endl;
        }
    }
}
```

笛卡尔树

```
//求连续区间最小值(也可以维护最大值)
vector<int>w(n + 1);
vector<int>ls(n + 1,0),rs(n + 1,0);//左子树 右子树
stack<int>stk;
for(int i = 1;i <= n;i++)
{
    int le = -1;//弹出链的根节点
    while(!stk.empty() && w[stk.top()] > w[i]){le = stk.top();stk.pop();}
    //把右链上大于w[i]的部分弹出来
    if(!stk.empty())rs[stk.top()] = i;//i链接到右链上
    if(le != -1)ls[i] = le;//弹出的链变成i的左子树
    stk.push(i);
}
//栈底为根节点
```

倍增求LCA

```
struct Graph
{
    struct edge
    {
        int nx;
        int to;
        int w;
    };

    std::vector<edge> e;
    std::vector<int> heads;

    Graph(int n = 0,int m = 0) : heads(n + 1,-1)
    {
        e.reserve(m);
    }

    void add(int u,int v,int w = 0)
    {
        e.push_back({heads[u],v,w});
        heads[u] = e.size() - 1;
    }
};
struct LCA : Graph
{
```

```cpp
    int root,n,LOG;
    std::vector<std::vector<int>>fa;
    std::vector<int>dep;

    LCA(int num,int rt = 1) : Graph(num,2 * std::max(0LL,num - 1))
    {
        n = num;
        root = rt;
        LOG = 1;
        while((1LL << LOG) <= n)LOG++;

        fa.assign(n + 1,std::vector<int>(LOG));
        dep.assign(n + 1,0);
    }

    void init(int u)
    {
        for(int i = 1;i < LOG;i++)
            fa[u][i] = fa[fa[u][i - 1]][i - 1];

        for(int i = heads[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(v == fa[u][0])continue;
            fa[v][0] = u;
            dep[v] = dep[u] + 1;
            init(v);
        }
    }
    void build()
    {
        init(root);
    }
    int lca(int x,int y)
    {
        if(dep[x] < dep[y])std::swap(x,y);
        int d = dep[x] - dep[y];
        for(int i = 0;d;i++)
        {
            if(d & 1)x = fa[x][i];
            d >>= 1;
        }
        if(x == y)return x;
        for(int i = LOG - 1;i >= 0;i--)
        {
            if(fa[x][i] != fa[y][i])
            {
                x = fa[x][i];
                y = fa[y][i];
            }
        }
        return fa[x][0];
    }
```

```
};
void solve()
{
    int n,rt;
    std::cin >> n >> rt;
    LCA g(n,rt);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g.add(u,v);
        g.add(v,u);
    }
    g.build();
    int x,y;
    std::cin >> x >> y;
    std::cout << g.lca(x,y) << "\n";
}
```

主席树

```
struct node
{
    int l;
    int r;
    int w;
}t[N << 5];
int a[N],tot = 0;
int cln(int p)//克隆
{
    t[++tot] = t[p];
    return tot;
}
int build(int l,int r)//建初始树
{
    int p = ++tot;
    if(l == r)
    {
        t[p].w = a[l];
        return p;
    }
    int mid = (l + r) >> 1;
    t[p].l = build(l,mid);
    t[p].r = build(mid + 1,r);
    return p;
}
int update(int p,int l,int r,int pos,int w)//返回复制的新节点
{
    int cl = cln(p);
    if(l == r)t[cl].w = w;
    else
    {
```

```
        int mid = (l + r) >> 1;
        if(pos <= mid)t[cl].l = update(t[p].l,l,mid,pos,w);
        else t[cl].r = update(t[p].r,mid + 1,r,pos,w);
    }
    return cl;
}
int query(int p,int l,int r,int pos)//查询
{
    if(l == r)return t[p].w;
    int mid = (l + r) >> 1;
    if(pos <= mid)return query(t[p].l,l,mid,pos);
    else return query(t[p].r,mid + 1,r,pos);
}
//使用方法
rt[M];//rt[i]为版本i的根节点
rt[0] = build(1,n);//可以不需要初始建树
rt[i] = update(rt[v],1,n,pos,w)//在版本v上将下标为pos的值更新为w，并返回新版本的根节点下标
query(rt[v],1,n,pos)//询问v版本下标为pos的值
```

主席树求静态区间第k小

```
struct node
{
    int l;
    int r;
    int cnt;
}t[N << 5];int tot = 0;
int cln(int p)
{
    t[++tot] = t[p];
    return tot;
}
int build(int l,int r)
{
    int p = ++tot;
    if(l == r)return p;
    int mid = (l + r) >> 1;
    t[p].l = build(l,mid);
    t[p].r = build(mid + 1,r);
    return p;
}
int update(int p,int l,int r,int pos)
{
    int cl = cln(p);t[cl].cnt++;
    if(l == r)return cl;
    int mid = (l + r) >> 1;
    if(pos <= mid)t[cl].l = update(t[p].l,l,mid,pos);
    else t[cl].r = update(t[p].r,mid + 1,r,pos);
    return cl;
};
int query(int u,int v,int l,int r,int k)
{
```

```
    if(l == r)return l;
    int mid = (l + r) >> 1;
    int tmp = t[t[v].l].cnt - t[t[u].l].cnt;
    if(k <= tmp)return query(t[u].l,t[v].l,l,mid,k);
    else return query(t[u].r,t[v].r,mid + 1,r,k - tmp);
}
void solve()
{
    int n,m;
    cin >> n >> m;
    vector<int>a(n + 1),b(n + 1),rt(n + 1);
    for(int i = 1;i <= n;i++)
    {
        cin >> a[i];
        b[i] = a[i];
    }
    //离散化
    sort(b.begin() + 1,b.end());
    b.erase(unique(b.begin() + 1,b.end()),b.end());
    int len = b.size();
    auto get_pos = [&](int w) -> int
    {
        return lower_bound(b.begin() + 1,b.end(),w) - b.begin();
    };
    rt[0] = build(1,len);
    for(int i = 1;i <= n;i++)
        rt[i] = update(rt[i - 1],1,len,get_pos(a[i]));
    while(m--)
    {
        int l,r,k;
        cin >> l >> r >> k;
        cout << b[query(rt[l - 1],rt[r],1,len,k)] << endl;
    }
}
```

分块

```
void solve()
{
    int n,m,u;
    cin >> n >> m >> u;
    int len = sqrt(n) + 1;//块长
    vector<int>a(n + 1),id(n + 1);//原数组，编号
    vector<vector<int>>b(len + 1);//块内数
    for(int i = 1;i <= n;i++)
    {
        cin >> a[i];
        id[i] = (i - 1) / len + 1;
        b[id[i]].push_back(a[i]);
    }
    for(int i = 1;i <= len;i++)sort(b[i].begin(),b[i].end());
    auto query = [&](int l,int r,int v) -> int
```

```
    {
        int lid = id[l],rid = id[r];
        int sum = 0;
        if(lid == rid)
        {
            for(int i = l;i <= r;i++)
                if(a[i] < v)sum++;
            return sum;
        }
        for(int i = l;id[i] == lid;i++)
            if(a[i] < v)sum++;
        for(int i = lid + 1;i < rid;i++)
            sum += lower_bound(b[i].begin(),b[i].end(),v) - b[i].begin();
        for(int i = r;id[i] == rid;i--)
            if(a[i] < v)sum++;
        return sum;
    };
    auto update = [&](int pos,int k,int l,int r) -> void
    {
        int iid = id[pos];
        b[iid].erase(lower_bound(b[iid].begin(),b[iid].end(),a[pos]));
        a[pos] = u * k / (r - l + 1);
        b[iid].push_back(a[pos]);
        sort(b[iid].begin(),b[iid].end());
    };
    while(m--)
    {
        int l,r,v,p;
        cin >> l >> r >> v >> p;
        int cnt = query(l,r,v);
        update(p,cnt,l,r);
    }
    for(int i = 1;i <= n;i++)
        cout << a[i] << endl;
}
```

二进制分块

可以将一个二进制数组按64位分块存储

一般用于查询时，要选择子数组的二进制并对其进行位运算操作

```
typedef unsigned long long ull;
ull A[N];
ull Bit(int l,int r)
{
    ull ret = (r == 63) ? -1ull : (1ull << (r + 1)) - 1;
    ret ^= (1ull << l) - 1;
    return ret;
}
struct Bitset
{
    vector<ull>arr;//按64分块存储
```

```
    int pos;
    int sz;
    Bitset(int _sz) {
        sz = _sz; // 位总长度
        pos = 0; // 当前添加位置
        arr.resize((_sz + 63) >> 6); // 按64位分块
    }
    void add(int cnt,ull val)//将长度为cnt的val加进来
    {
        if(cnt <= 64 - (pos & 63))
            arr[pos >> 6] |= val << (pos & 63);
        else
        {
            ull mask = Bit(0,64 - (pos & 63) - 1);
            arr[pos >> 6] |= (val & mask) << (pos & 63);
            arr[(pos >> 6) + 1] = val >> (64 - (pos & 63));
        }
        pos += cnt;
    }
    ull calc()//对于每一块进行答案统计
    {
        ull ret = 0;
        int ra = 0;
        for(int i = 0;i < arr.size();i++)
        {

        }
        return ret;
    }
};
Bitset getBitset(int l,int r)//将原数组[l,r]取出来
{
    Bitset res(r - l + 1);
    if((l >> 6) == (r >> 6))
    {
        ull val = (A[l >> 6] & Bit(l & 63,r & 63)) >> (l & 63);
        res.add(r - l + 1,val);
    }
    else
    {
        ull val = (A[l >> 6] & Bit(l & 63,63)) >> (l & 63);
        res.add(63 - (l & 63) + 1,val);
        for(int i = (l >> 6) + 1;i < (r >> 6);i++)
            res.add(64,A[i]);
        val = A[r >> 6] & Bit(0,r & 63);
        res.add((r & 63) + 1,val);
    }
    return res;
}

void solve()
{
    //初始化方法
```

```
    string s;cin >> s;
    for(int i = 0;i < n;i += 64)
    {
        ull val = 0;
        for(int j = 0;j < 64 && j + i < n;j++)
            val |= (ull)(s[i + j] - '0') << j;
        A[i >> 6] = val;
    }
}
```

珂朵莉树

```
//将值相同的一段区间合并进行维护
//每次遍历后更新区间(区间被合并成一个节点),时间复杂度mlog(n),否则复杂度容易爆
//也可以用于保证数据随机的题目
struct Chtholly_Tree
{
    struct node
    {
        int l;
        int r;
        mutable int v;//可直接操作
        bool operator<(const node &tp)const{
            return l < tp.l;
        }
    };
    set<node>s;
    Chtholly_Tree(int n)
    {
        s.insert({1,n + 1,0});
    }
    auto spilt(int x)//将[l,r]分裂成[l,x),[x,r]
    {
        auto it = s.lower_bound(node{x,0,0});
        if(it != s.end() && it->l == x)return it;
        --it;
        int l = it->l,r = it->r,v = it->v;
        s.erase(it);
        s.insert({l,x - 1,v});
        return s.insert({x,r,v}).first;
    }
    void assign(int l,int r,int v)//将[l,r]赋值为v
    {
        auto itr = spilt(r + 1),itl = spilt(l);
        s.erase(itl,itr);
        s.insert({l,r,v});
    }
    void perform(int l,int r)//对[l,r]进行操作
    {
        auto itr = spilt(r + 1),itl = spilt(l);
        for(;itl != itr;++itl)
        {
```

```
            //work

        }
    }
};
```

动态规划

数位dp

```
int dp[N][11],up[N];

// pos: 当前处理的数位
// pre: 上一个有效数字，10表示不存在
// li: 高位前缀是否等于上界
// le: 是否仍处于前导零阶段
int dfs(int pos,int pre,bool li,bool le)
{
    if(pos == 0)return !le;
    if(!li && !le && dp[pos][pre] != -1)return dp[pos][pre];
    int mx = li ? up[pos] : 9;
    int ans = 0;
    for(int d = 0;d <= mx;d++)
    {
        // 题目约束：不能有相邻相同数字
        if(!le && d == pre)continue;

        bool nlimit = li && (d == up[pos]);
        bool nlead = le && (d == 0);
        int npre = nlead ? 10 : d;
        ans += dfs(pos - 1,npre,nlimit,nlead);
    }
    if(!li && !le)dp[pos][pre] = ans;
    return ans;
}
// 统计 [1,n] 中满足要求的整数数量
int calc(int n)
{
    if(n <= 0)return 0;
    std::memset(dp,-1,sizeof(dp));
    int len = 0;
    while(n)
    {
        up[++len] = n % 10;
        n /= 10;
    }
    return dfs(len,10,1,1);
}
void solve()
{
    int l,r;
    std::cin >> l >> r;
```

```
    std::cout << calc(r) - calc(l - 1) << "\n";
}
//数字特别大时
int calc(string s)//
{
    memset(dp,-1,sizeof(dp));
    reverse(s.begin(),s.end());
    int len = 0;
    for(char c : s)
        up[++len] = c - '0';
    return dfs(len,10,10,1,0,0);
}
```

矩阵快速幂
高斯消元
线性基
gcd,exgcd
中国剩余定理（CRT,exCRT）
Lucas定理
exLucas定理
数论分块
线性求逆元
组合数
三维前缀和
线性筛
欧拉函数
求一个数的最大质因子(1e18)
卡特兰数
快速傅里叶变换(FFT)
快速数论变换(NTT)
快速沃尔什变换(FWT)
多项式模板
取模类(int)

数学

矩阵快速幂

```
struct mat
{
    std::vector<std::vector<int>>a;
    mat(std::vector<std::vector<int>> &arr):a(arr){}
    mat operator*(const mat &tp)
    {
        auto &b = tp.a;
        int n = a.size(),m = b[0].size(),t = b.size();
        std::vector arr(n,std::vector<int>(m));
        for(int i = 0;i < n;i++)
            for(int j = 0;j < m;j++)
                for(int k = 0;k < t;k++)
                    arr[i][j] += a[i][k] * b[k][j];
        return mat(arr);
    }
    mat qmi(mat a,int b)
    {
        int n = a.a.size();
        std::vector arr(n,std::vector<int>(n));
        for(int i = 0;i < n;i++)arr[i][i] = 1;
        mat res(arr);
        while(b)
        {
            if(b & 1)res = res * a;
```

```
            a = a * a;
            b >>= 1;
        }
        return res;
    }
};

```

高斯消元

```
int gauss(std::vector<std::vector<int>> &arr)//为每一行找一个主元
{
    int lr = arr.size();
    int lc = arr[0].size();
    int row = 0;
    for(int col = 0;col < lr && row < lr;col++)
    {
        int tpr = row;
        for(int i = row;i < lr;i++)
            if(std::fabs(arr[tpr][col]) < std::fabs(arr[i][col]))tpr = i;
        if(std::fabs(arr[tpr][col]) < eps)continue;
        if(tpr != row)
            std::swap(arr[tpr],arr[row]);
        double w = arr[row][col];
        for(int j = col;j < lc;j++)arr[row][j] /= w;
        for(int i = 0;i < lr;i++)
        {
            if(i == row)continue;
            w = arr[i][col];
            for(int j = col;j < lc;j++)
                arr[i][j] -= w * arr[row][j];
        }
        row++;
    }
    return row;
}
void solve()
{
    int n;std::cin >> n;
    std::vector a(n,std::vector<double>(n + 1));
    for(int i = 0;i < n;i++)
        for(int j = 0;j <= n;j++)std::cin >> a[i][j];
    int rk = gauss(a);
    if(rk < n)std::cout << "No Solution\n";
    else
        for(int i = 0;i < n;i++)
            std::cout << std::fixed << std::setprecision(2) << a[i][n] << "\n";
}
```

线性基

```

```

```cpp
struct LBasis
{
    static constexpr int LOG = 62;
    int p[LOG + 1],rk;
    LBasis(int n = LOG + 1)
    {
        rk = 0;
        std::memset(p,0,sizeof(int) * n);
    }

    bool insert(long long x,int op = 1)//op = 1,插入
    {
        for(int i = LOG;i >= 0;i--)
        {
            if (!(x >> i & 1)) continue;
            if (!p[i])
            {
                if(op)
                {
                    p[i] = x;
                    rk++;
                }
                return true;
            }
            x ^= p[i];
        }
        return false;
    }
    int Qmx(int x = 0)
    {
        int ans = x;
        for(int i = LOG;i >= 0;i--)
            ans = std::max(ans,ans ^ p[i]);
        return ans;
    }
};
```

gcd, exgcd

```cpp
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
int exgcd(int a, int b, int &x, int &y)
{
    if (!b)
    {
        x = 1;
        y = 0;
        return a;
    }
    int d = exgcd(b, a % b, x, y);
    int t = x;
    x = y;
    y = t - (a / b) * y;
```

```
    return d;
}
```

中国剩余定理（CRT, exCRT）

```
namespace CRT
{
    int exgcd(int a, int b, int &x, int &y)
    {
        if (!b)
        {
            x = 1;
            y = 0;
            return a;
        }
        int d = exgcd(b, a % b, x, y);
        int t = x;
        x = y;
        y = t - (a / b) * y;
        return d;
    }
    //返回通解 x === ai mod mi [0,n) m 两两互质
    pii CRT(int n,std::vector<int> &a,std::vector<int> &m)
    {
        int M = 1;
        for(int i = 0;i < n;i++)M = M * m[i];
        i128 ans = 0;
        for(int i = 0;i < n;i++)
        {
            int w = M / m[i];
            int inv,y;inv = y = 0; // The inv of w
            exgcd(w,m[i],inv,y);
            inv = (inv % m[i] + m[i]) % m[i];
            ans += i128(a[i]) * w % M * inv % M;
            ans %= M;
        }
        return {ans,M};
    }
    //返回通解 x === ai mod mi [0,n) m 两两不一定互质,{-1,-1} 为无解
    pii exCRT(int n,std::vector<int> &a,std::vector<int> &m)
    {
        int a1 = a[0],m1 = m[0];
        for(int i = 1;i < n;i++)
        {
            int x,y;
            int g = exgcd(m1,m[i],x,y);//gcd(m1,m[i])
            if((a[i] - a1) % g != 0)return {-1,-1};
            i128 P = (i128)x * ((a[i] - a1) / g);//特解，没有限制正负

            int M = m1 / g * m[i];//新的 m1

            a1 = (P * m1 % M + a1) % M;//新的 a1 = m1 * x + a1
```

```
            a1 = (a1 % M + M) % M;
            m1 = M;
        }
        return {a1,m1};
    }
};
```

Lucas定理

```
struct Lucas
{
    int p;//p <= 1e7 && isprime;

    std::vector<int>fa,infa;
    Lucas(int p):p(p),fa(p),infa(p)
    {
        fa[0] = 1;
        for(int i = 1;i < p;i++)fa[i] = fa[i - 1] * i % p;
        infa[p - 1] = qmi(fa[p - 1],p - 2);
        for(int i = p - 1;i >= 1;i--)infa[i - 1] = infa[i] * i % p;
    }
    int qmi(int a,int b)
    {
        int res = 1;
        while(b)
        {
            if(b & 1)res = res * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return res;
    }
    int C(int n,int m)
    {
        if(n < m)return 0;
        return fa[n] * infa[m] % p * infa[n - m] % p;
    }
    int lucas(int n,int m)
    {
        if(n < p && m < p)return C(n,m);
        return C(n % p,m % p) * lucas(n / p,m / p) % p;
    }
};
Lucas lucas(p);
lucas.lucas(n,m);
```

exLucas定理

```cpp
struct exLucas
{
    struct node
    {
        int p,k,q;
        std::vector<int>c,s,f,f_inv;
        bool fl;
        int qmi(int a,int b)
        {
            int res = 1;
            while(b)
            {
                if(b & 1)res = res * a % q;
                a = a * a % q;
                b >>= 1;
            }
            return res;
        }
        node(int p,int k,int q,int n) : p(p),k(k),q(q),s(q),fl(0)
        {
            s[0] = 1;
            for(int i = 1;i < q;i++)
                s[i] = s[i - 1] * (i % p ? i : 1) % q;//分块，前 i 项积
            if(n <= N)
            {
                fl = 1;
                c.resize(n + 1);
                f.resize(n + 1);
                f_inv.resize(n + 1);
                f[0] = 1;
                for(int i = 1;i <= n;i++)
                {
                    c[i] = c[i - 1];
                    int cnt = i;
                    while(cnt % p == 0)c[i]++,cnt /= p;
                    f[i] = f[i - 1] * cnt % q;
                }

                f_inv[n] = qmi(f[n],q - q/p - 1);//phi(q) - 1
                for(int i = n;i >= 1;i--)
                {
                    int x = i;
                    while(x % p == 0)x /= p;
                    f_inv[i - 1] = f_inv[i] * x % q;
                }
            }
        }
        int F_c(int n)
        {
            int ans = 0;
            while(n)
            {
                ans += n / p;
```

```
            n /= p;
        }
        return ans;
    }
    int F_f(int n)
    {
        if(n == 0)return 1;
        int ans = qmi(s[q - 1],n / q) * s[n % q] % q * F_f(n / p) % q;
        return ans;
    }
    int F_f_inv(int n)
    {
        int ans = F_f(n);
        return qmi(ans,q - q / p - 1);
    }

    //n <= N,使用预处理，否则现算
    pii exlucas(int n,int m)
    {
        if(m > n)return {0,q};
        if(fl)
        {
            int cnt = c[n] - c[m] - c[n - m];
            int ans = qmi(p,cnt);
            int tmp = f[n] * f_inv[n - m] % q * f_inv[m] % q;
            return {ans * tmp % q,q};
        }
        else
        {
            int cnt = F_c(n) - F_c(m) - F_c(n - m);
            int ans = qmi(p,cnt);
            int tmp = F_f(n) * F_f_inv(n - m) % q * F_f_inv(m) % q;
            return {ans * tmp % q,q};
        }
    }
};
int M,siz;//M <= N;
std::vector<node>pri;//每个质因数预处理
//1 ~ n 范围组合数，对 M 取余
exLucas(int m,int n) : M(m),siz(0)
{
    while(m != 1)
    {
        int p = minp[m],k = 0,q = 1;
        while(minp[m] == p)
        {
            k++;
            q *= p;
            m /= p;
        }
        pri.push_back(node{p,k,q,n});
        siz++;
    }
```

```
    }
    int C(int n,int m)
    {
        if(m > n)return 0;
        std::vector<int>a(siz),b(siz);
        for(int i = 0;i < siz;i++)
        {
            pii ans = pri[i].exlucas(n,m);
            a[i] = ans.first;
            b[i] = ans.second;
        }
        return CRT::CRT(siz,a,b).first;
    }
};
```

数论分块

```
int k,l = 1,r,n;
//向下分块(l~r,tmp相等)
while(l <= n)
{
    tmp = n / l;
    r = n / tmp;
    k = tmp;
    l = r + 1;
}
//向上分块(l ~ r,k相等)
while(l <= n)
{
    tmp = (n - 1) / l;
    r = (n - 1) / tmp;//注意分母为0的情况(l == n)
    k = (n + l - 1) / l;

    l = r + 1;
}
//n维数论分块
```

线性求逆元

```
int n = 1e7;
vector<int>inv(n + 1);
for(int i = 2;i <= n;i++)inv[i] = (mod - mod / i) * inv[mod % i] % mod;
```

组合数

```
//普通预处理(<=2000)
constexpr int N = 2010;
constexpr int P = 1e9+7;
```

```cpp
int C[N][N];
void init()
{
    for(int i = 0; i < N;i++)
        for(int j = 0; j <= i;j++)
            if(!j)C[i][j] = 1;
            else C[i][j] = (C[i - 1][j] + C[i - 1][j - 1]) % P;
}

//预处理阶乘+逆元(<=1e5)
constexpr int N = 2e5 + 7;
constexpr int P = 998244353;
int fa[N],infa[N];
int qmi(int a,int b)
{
    int res = 1;
    while(b)
    {
        if(b & 1)res = res * a % P;
        a = a * a % P;
        b >>= 1;
    }
    return res;
}
void init()
{
    fa[0] = 1;
    for(int i = 1;i < N;i++)fa[i] = fa[i - 1] * i % P;
    infa[N - 1] = qmi(fa[N - 1],P - 2);
    for(int i = N - 1;i >= 1;i--)infa[i - 1] = infa[i] * i % P;
}
int C(int a,int b)
{
    if(b > a)return 0;
    return fa[a] * infa[b] % P * infa[a - b] % P;
}

//卢卡斯定理(a,b<=1e18,询问次数<=20,模数p<=1e5)
int qmi(int a, int k, int p) {
    int res = 1 % p;
    while (k) {
        if (k & 1) res = res * a % p;
        a = a * a % p;
        k >>= 1;
    }
    return res;
}
// 定义求解
int C(int a, int b, int p) {
    if (b > a) return 0;
    int res = 1;
    for (int i = 1, j = a; i <= b; ++i, --j) {
        res = res * j % p;
```

```
        res = res * qmi(i, p - 2, p) % p;
    }
    return res;
}

int lucas(int a, int b, int p) {
    if (a < p && b < p) return C(a, b, p);
    return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}
void print(int a,int b,int c)
{
    lucas(a, b, p);
}
```

三维前缀和

```
//容斥原理
for (int i = 0; i <= 150; i++)
{
    for (int j = 0; j <= 150; j++)
    {
        for (int k = 0; k <= 150; k++)
        {
            if(j-1>=0)ans[i][j][k] += ans[i][j - 1][k]; //后方
            if(i-1>=0)ans[i][j][k] += ans[i - 1][j][k];//左侧
            if(k-1>=0)ans[i][j][k] += ans[i][j][k - 1]; //下方
            if (i - 1 >= 0 && j - 1 >= 0)ans[i][j][k] -= ans[i - 1][j - 1][k] - mod;//左后
            if (i - 1 >= 0 && k - 1 >= 0)ans[i][j][k] -= ans[i - 1][j][k - 1]-mod; //左下
            if (k - 1 >= 0 && j - 1 >= 0)ans[i][j][k] -= ans[i][j - 1][k - 1]-mod; //后下
            if (i - 1 >= 0 && j - 1 >= 0&&k-1>=0)ans[i][j][k] += ans[i - 1][j - 1][k - 1];
            ans[i][j][k] %= mod;
        }
    }
}
//查询
int q;cin >> q;
while(q--)
{
    int lx,rx,ly,ry,lz,rz;
    cin >> lx >> rx >> ly >> ry >> lz >> rz;
    int sum = ans[rx][ry][rz];
    lx--,ly--,lz--;
    sum -= ans[rx][ry][lz] + ans[rx][ly][rz] + ans[lx][ry][rz];
    sum += ans[rx][ly][lz] + ans[lx][ry][lz] + ans[lx][ly][rz];
    sum -= ans[lx][ly][lz];
    cout << sum << endl;
}
```

线性筛

```cpp
//筛素数
std::vector<int>pri;
bool vis[N];
int minp[N];//存数的最小质因子
void init()
{
    for(int i = 2;i < N;i++)
    {
        if(!vis[i]){pri.push_back(i);minp[i] = i;}
        for(auto x : pri)
        {
            if(i * x >= N)break;
            vis[i * x] = 1;
            minp[i * x] = x;
            if(i % x == 0)break;
        }
    }
}
//线性筛 求欧拉函数
vector<int>pri;
bool vis[N];
int phi[N];
void init()
{
    phi[1] = 1;
    for(int i = 2;i < N;i++)
    {
        if(!vis[i])
        {
            pri.push_back(i);
            phi[i] = i - 1;
        }
        for(auto x:pri)
        {
            if(x * i >= N)break;
            vis[i * x] = 1;
            if(i % x == 0)
            {
                phi[i * x] = x * phi[i];
                break;
            }
            else phi[i * x] = phi[i] * phi[x];
        }
    }
}
//线性筛 求莫比乌斯函数
vector<int>pri;
bool vis[N];
int mu[N];
void init()
```

```
{
    mu[1] = 1;
    for(int i = 2;i < N;i++)
    {
        if(!vis[i])
        {
            mu[i] = -1;
            pri.push_back(i);
        }
        for(int v : pri)
        {
            if(i * v >= N)break;
            vis[i * v] = 1;
            if(i % v == 0)
            {
                mu[i * v] = 0;
                break;
            }
            mu[i * v] = -mu[i];
        }
    }
}
```

欧拉函数

```
//phi(n) = n * (pi - 1) / pi;
//pi 为 n 的所有质因数集合
int euler_phi(int n)
{
    int ans = n;
    for(int i = 2;i * i <= n;i++)
        if (n % i == 0)
        {
            ans = ans / i * (i - 1);
            while(n % i == 0)n /= i;
        }
    if(n > 1)ans = ans / n * (n - 1);
    return ans;
}
```

求一个数的最大质因子(1e18)

```
int mul(int a,int b,int m)
{
    ull c = (ull)a * (ull)b - (ull)((long double)a / m * b + 0.5L) * (ull)m;
    if(c < (ull)m)return c;
    return c + m;
}
int qmi(int a,int b,int p)
{
    int res = 1;
```

```
    while(b)
    {
        if(b & 1)res = mul(res,a,p);
        b >>= 1;
        a = mul(a,a,p);
    }
    return res;
}
bool Miller_Rabin(int p)//判是否是素数
{
    if(p < 2)return false;
    if(p == 2 || p == 3)return true;
    int d = p - 1,r = 0;
    while(!(d & 1))r++,d >>= 1;  // 将d处理为奇数
    for(int k = 0;k < 10;k++)
    {
        int a = rand() % (p - 2) + 2;
        int x = qmi(a, d, p);
        if (x == 1 || x == p - 1) continue;
        for(int i = 0;i < r - 1;i++)
        {
            x = mul(x,x,p);
            if(x == p - 1)break;
        }
        if (x != p - 1) return false;
    }
    return true;
}
int Pollard_Rho(int x)//取x的一个因数
{
    int s = 0, t = 0;
    int c = (int)rand() % (x - 1) + 1;
    int step = 0,goal = 1;
    int val = 1;
    for(goal = 1;;goal <<= 1,s = t,val = 1)
    {
        for(step = 1;step <= goal;step++)
        {

            t = (mul(t,t,x) + c) % x;
            val = mul(val,llabs(t - s),x);
            if(step % 127 == 0)
            {
                int d = gcd(val,x);
                if(d > 1)return d;
            }
        }
        int d = gcd(val,x);
        if(d > 1)return d;
    }
}
int mx;
void fac(int x)
```

```
{
    if(x <= mx || x < 2)return;
    if(Miller_Rabin(x))
    {
        mx = max(mx,x);
        return;
    }
    int p = x;
    while(p >= x)p = Pollard_Rho(x);
    while(x % p == 0)x /= p;
    fac(x), fac(p);  // 继续向下分解x和p
}
void solve()
{
    srand((unsigned)time(NULL));
    int n;
    cin >> n;
    mx = 0;
    fac(n);
    if(mx == n)cout << "Prime" << endl;
    else cout << mx << endl;
}
```

卡特兰数

```
int Ca(int n)
{
    if(n<=1) return 1;
    vector<int>f(n + 1);
    f[0] = f[1] = 1;
    for(int i = 1;i <= n;i++)
    {
        f[i] = 0;
        for(int j = 0;j < i;j++)
            f[i] += (f[i] * f[i - j - 1]);
    }
    return f[n];
}
//Ca[n] = Ca[n - 1] * (4 * n - 2) / (n + 1);
//Ca[n] = C(2n,n) - C(2n,n - 1);
//Ca[n] = Ca[i] * Ca[n - i - 1] (i,1,n);
```

快速傅里叶变换(FFT)

```
typedef complex<double> cp;
const double pi = acos(-1);
vector<int>rev;//位逆序置换
void fft(vector<cp> &a,int flag = 1)//1为DFT,-1为IDFT
{
```

```
    int n = a.size();
    if(rev.size() != n)
    {
        rev.resize(n);
        int k = __builtin_ctz(n) - 1;
        for(int i = 0;i < n;i++)
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
    }
    for(int i = 0;i < n;i++)
        if(rev[i] < i)swap(a[i],a[rev[i]]);
    for(int len = 1;len < n;len *= 2)
    {
        cp wn = exp(cp(0,flag * pi / len));//单位根
        for(int i = 0;i < n;i += 2 * len)//合并每一组
        {
            cp w = cp(1,0);//w0
            for(int j = 0;j < len;j++)//合并
            {
                cp x = a[i + j];
                cp y = a[i + j + len] * w;
                a[i + j] = x + y;
                a[i + j + len] = x - y;
                w *= wn;//wi + 1;
            }
        }
    }
    if(flag == -1)
        for(int i = 0;i < n;i++)
            a[i] /= n;
}
vector<cp> mul(vector<cp> a,vector<cp> b)//求卷积
{
    int n = 1,tot = a.size() + b.size() - 1;
    while(n < tot)n <<= 1;
    a.resize(n);
    b.resize(n);
    fft(a);
    fft(b);
    for(int i = 0;i < n;i++)a[i] = a[i] * b[i];
    fft(a,-1);
    a.resize(tot);//x ^ (tot - 1)
    return a;
}

//应用1:高精度乘法
void solve()
{
    string s1,s2;
    cin >> s1 >> s2;
    int len1 = s1.size();
    int len2 = s2.size();
    vector<cp> a(len1),b(len2);
    for(int i = 0;i < len1;i++)a[i] = (double)(s1[len1 - 1 - i] - '0');
```

```
    for(int i = 0;i < len2;i++)b[i] = (double)(s2[len2 - 1 - i] - '0');
    a = mul(a,b);
    int len = a.size();
    vector<int>ans(len + 1);//多开1位存进位
    for(int i = 0;i < len;i++)
    {
        ans[i] += (int)(a[i].real() + 0.5);
        ans[i + 1] += ans[i] / 10;
        ans[i] %= 10;
    }
    while(len > 0 && !ans[len])len--;
    for(;len >= 0;len--)cout << ans[len];
    cout << endl;
}
```

快速数论变换(NTT)

```
const int mod = 998244353;
int qmi(int a,int b)
{
    int res = 1;
    while(b)
    {
        if(b & 1)res = res * a % mod;
        b >>= 1;
        a = a * a % mod;
    }
    return res;
}
vector<int>rev;
void ntt(vector<int> &a,int flag = 1)
{
    int n = a.size();
    if(rev.size() != n)
    {
        rev.resize(n);
        int k = __builtin_ctz(n) - 1;
        for(int i = 0;i < n;i++)
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
    }
    for(int i = 0;i < n;i++)
        if(rev[i] < i)swap(a[i],a[rev[i]]);
    for(int len = 1;len < n;len *= 2)
    {
        int gn = qmi(3,(mod - 1) / (len << 1));//原根
        for(int i = 0;i < n;i += 2 * len)
        {
            int g = 1;
            for(int j = 0;j < len;j++)
            {
                int x = a[i + j];
                int y = a[i + j + len] * g % mod;
```

```
                a[i + j] = (x + y) % mod;
                a[i + j + len] = (x - y + mod) % mod;
                g = g * gn % mod;
            }
        }
    }
    if(flag == -1)
    {
        reverse(a.begin() + 1,a.end());//多个交换
        int inv = qmi(n,mod - 2);
        for(int i = 0;i < n;i++)
            a[i] = a[i] * inv % mod;
    }
}
vector<int> mul(vector<int> a,vector<int> b)
{
    int n = 1,tot = a.size() + b.size() - 1;
    while(n < tot) n <<= 1;
    a.resize(n);
    b.resize(n);
    ntt(a);
    ntt(b);
    for(int i = 0;i < n;i++)
        a[i] = a[i] * b[i] % mod;
    ntt(a,-1);
    a.resize(tot);
    return a;
}

//高精乘法
void solve()
{
    string s1,s2;
    cin >> s1 >> s2;
    int len1 = s1.size();
    int len2 = s2.size();
    vector<int>a(len1),b(len2);
    for(int i = 0;i < len1;i++)a[i] = s1[len1 - 1 - i] - '0';
    for(int i = 0;i < len2;i++)b[i] = s2[len2 - 1 - i] - '0';
    a = mul(a,b);
    int len = a.size();
    vector<int>ans(len + 1);
    for(int i = 0;i < len;i++)
    {
        ans[i] += a[i];
        ans[i + 1] += ans[i] / 10;
        ans[i] %= 10;
    }
    while(len > 0 && !ans[len])len--;
    for(;len >= 0;len--)cout << ans[len];
    cout << endl;
}
```

快速沃尔什变换(FWT)

```
int qmi(int a,int b)
{
    int res = 1;
    while(b)
    {
        if(b & 1)res = res * a % mod;
        b >>= 1;
        a = a * a % mod;
    }
    return res;
}
int add(int u,int v)
{
    return (u + v + mod) % mod;
}
int mul(int u,int v)
{
    return u * v % mod;
}
void fwt_or(vector<int>&a,int flag = 1)//-1为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
                a[i + j + len] = add(a[i + j + len],mul(flag,a[i + j]));
    return;
}
void fwt_and(vector<int>&a,int flag = 1)//-1为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
                a[i + j] = add(a[i + j],mul(flag,a[i + j + len]));
    return;
}
void fwt_xor(vector<int>&a,int flag = 1)//1/2为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
            {
                a[i + j] = add(a[i + j],a[i + j + len]);
                a[i + j + len] = add(a[i + j],-mul(2,a[i + j + len]));
                a[i + j] = mul(a[i + j],flag);
                a[i + j + len] = mul(a[i + j + len],flag);
            }
    return;
```

```cpp
}
void fwt_xnor(vector<int>&a,int flag = 1)//1/2为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
            {
                a[i + j + len] = add(a[i + j + len],a[i + j]);
                a[i + j] = add(a[i + j + len],-mul(2,a[i + j]));
                a[i + j + len] = mul(a[i + j + len],flag);
                a[i + j] = mul(a[i + j],flag);
            }
    return;
}
//应用：卷积运算
vector<int> work(vector<int>a,vector<int>b)
{
    int n = a.size();
    fwt(a);
    fwt(b);
    for(int i = 0;i < n;i++)a[i] = mul(a[i],b[i]);
    fwt(a,mod - 1);//or,and
    fwt(a,qmi(2,mod - 2))//xor,xnor
    return a;
}
```

多项式模板

```cpp
using u64 = unsigned long long;
using i64 =  long long;
using u32 = unsigned;

template<class T>
constexpr T power(T a, u64 b, T res = 1) {
    for (; b != 0; b /= 2, a *= a) {
        if (b & 1) {
            res *= a;
        }
    }
    return res;
}

template<u32 P>
struct ModIntBase {
public:
    constexpr ModIntBase() : x(0) {}
    template<class T>
    constexpr ModIntBase(T x_) {
        T v = x_ % T(mod());
        if (v < 0) {
            v += mod();
```

```
        }
        x = v;
    }

    constexpr static u32 mod() {
        return P;
    }

    constexpr u32 val() const {
        return x;
    }

    constexpr u32 mulMod(u32 a, u32 b) {
        return u64(a) * b % mod();
    }

    constexpr ModIntBase operator-() const {
        ModIntBase res;
        res.x = (x == 0 ? 0 : mod() - x);
        return res;
    }

    constexpr ModIntBase inv() const {
        return power(*this, mod() - 2);
    }

    constexpr ModIntBase &operator*=(const ModIntBase &rhs) & {
        x = mulMod(x, rhs.val());
        return *this;
    }
    constexpr ModIntBase &operator+=(const ModIntBase &rhs) & {
        x += rhs.val();
        if (x >= mod()) {
            x -= mod();
        }
        return *this;
    }
    constexpr ModIntBase &operator-=(const ModIntBase &rhs) & {
        x -= rhs.val();
        if (x >= mod()) {
            x += mod();
        }
        return *this;
    }
    constexpr ModIntBase &operator/=(const ModIntBase &rhs) & {
        return *this *= rhs.inv();
    }

    friend constexpr ModIntBase operator*(ModIntBase lhs, const ModIntBase &rhs) {
        lhs *= rhs;
        return lhs;
    }
    friend constexpr ModIntBase operator+(ModIntBase lhs, const ModIntBase &rhs) {
```

```
        lhs += rhs;
        return lhs;
    }
    friend constexpr ModIntBase operator-(ModIntBase lhs, const ModIntBase &rhs) {
        lhs -= rhs;
        return lhs;
    }
    friend constexpr ModIntBase operator/(ModIntBase lhs, const ModIntBase &rhs) {
        lhs /= rhs;
        return lhs;
    }

    friend constexpr std::istream &operator>>(std::istream &is, ModIntBase &a) {
        i64 i;
        is >> i;
        a = i;
        return is;
    }
    friend constexpr std::ostream &operator<<(std::ostream &os, const ModIntBase &a) {
        return os << a.val();
    }

    friend constexpr bool operator==(const ModIntBase &lhs, const ModIntBase &rhs) {
        return lhs.val() == rhs.val();
    }

private:
    u32 x;
};

template<u32 P>
using ModInt = ModIntBase<P>;

//多项式部分,以上为取模板子
constexpr int P = 998244353;
using Z = ModInt<P>;
std::vector<int> rev;
std::vector<Z> roots{0, 1};
void dft(std::vector<Z> &a) {
    int n = a.size();

    if (int(rev.size()) != n) {
        int k = __builtin_ctz(n) - 1;
        rev.resize(n);
        for (int i = 0; i < n; i++) {
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
        }
    }

    for (int i = 0; i < n; i++) {
        if (rev[i] < i) {
            std::swap(a[i], a[rev[i]]);
        }
```

```
    }
    if (int(roots.size()) < n) {
        int k = __builtin_ctz(roots.size());
        roots.resize(n);
        while ((1 << k) < n) {
            Z e = power(Z(3), (P - 1) >> (k + 1));
            for (int i = 1 << (k - 1); i < (1 << k); i++) {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * e;
            }
            k++;
        }
    }
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                Z u = a[i + j];
                Z v = a[i + j + k] * roots[k + j];
                a[i + j] = u + v;
                a[i + j + k] = u - v;
            }
        }
    }
}
void idft(std::vector<Z> &a) {
    int n = a.size();
    std::reverse(a.begin() + 1, a.end());
    dft(a);
    Z inv = (1 - P) / n;
    for (int i = 0; i < n; i++) {
        a[i] *= inv;
    }
}
struct Poly {
    std::vector<Z> a;
    Poly() {}
    explicit Poly(int size, std::function<Z(int)> f = [](int) { return 0; }) : a(size)
{
        for (int i = 0; i < size; i++) {
            a[i] = f(i);
        }
    }
    Poly(const std::vector<Z> &a) : a(a) {}
    Poly(const std::initializer_list<Z> &a) : a(a) {}
    int size() const {
        return a.size();
    }
    void resize(int n) {
        a.resize(n);
    }
    Z operator[](int idx) const {
        if (idx < size()) {
            return a[idx];
```

```
        } else {
            return 0;
        }
    }
    Z &operator[](int idx) {
        return a[idx];
    }
    Poly mulxk(int k) const {
        auto b = a;
        b.insert(b.begin(), k, 0);
        return Poly(b);
    }
    Poly modxk(int k) const {
        k = std::min(k, size());
        return Poly(std::vector<Z>(a.begin(), a.begin() + k));
    }
    Poly divxk(int k) const {
        if (size() <= k) {
            return Poly();
        }
        return Poly(std::vector<Z>(a.begin() + k, a.end()));
    }
    friend Poly operator+(const Poly &a, const Poly &b) {
        std::vector<Z> res(std::max(a.size(), b.size()));
        for (int i = 0; i < int(res.size()); i++) {
            res[i] = a[i] + b[i];
        }
        return Poly(res);
    }
    friend Poly operator-(const Poly &a, const Poly &b) {
        std::vector<Z> res(std::max(a.size(), b.size()));
        for (int i = 0; i < int(res.size()); i++) {
            res[i] = a[i] - b[i];
        }
        return Poly(res);
    }
    friend Poly operator-(const Poly &a) {
        std::vector<Z> res(a.size());
        for (int i = 0; i < int(res.size()); i++) {
            res[i] = -a[i];
        }
        return Poly(res);
    }
    friend Poly operator*(Poly a, Poly b) {
        if (a.size() == 0 || b.size() == 0) {
            return Poly();
        }
        if (a.size() < b.size()) {
            std::swap(a, b);
        }
        if (b.size() < 128) {
            Poly c(a.size() + b.size() - 1);
            for (int i = 0; i < a.size(); i++) {
```

```
                for (int j = 0; j < b.size(); j++) {
                    c[i + j] += a[i] * b[j];
                }
            }
            return c;
        }
        int sz = 1, tot = a.size() + b.size() - 1;
        while (sz < tot) {
            sz *= 2;
        }
        a.a.resize(sz);
        b.a.resize(sz);
        dft(a.a);
        dft(b.a);
        for (int i = 0; i < sz; ++i) {
            a.a[i] = a[i] * b[i];
        }
        idft(a.a);
        a.resize(tot);
        return a;
    }
    friend Poly operator*(Z a, Poly b) {
        for (int i = 0; i < int(b.size()); i++) {
            b[i] *= a;
        }
        return b;
    }
    friend Poly operator*(Poly a, Z b) {
        for (int i = 0; i < int(a.size()); i++) {
            a[i] *= b;
        }
        return a;
    }
    Poly &operator+=(Poly b) {
        return (*this) = (*this) + b;
    }
    Poly &operator-=(Poly b) {
        return (*this) = (*this) - b;
    }
    Poly &operator*=(Poly b) {
        return (*this) = (*this) * b;
    }
    Poly &operator*=(Z b) {
        return (*this) = (*this) * b;
    }
    Poly deriv() const {
        if (a.empty()) {
            return Poly();
        }
        std::vector<Z> res(size() - 1);
        for (int i = 0; i < size() - 1; ++i) {
            res[i] = (i + 1) * a[i + 1];
        }
```

```
        return Poly(res);
    }
    Poly integr() const {
        std::vector<Z> res(size() + 1);
        for (int i = 0; i < size(); ++i) {
            res[i + 1] = a[i] / (i + 1);
        }
        return Poly(res);
    }
    Poly inv(int m) const {
        Poly x{a[0].inv()};
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x * (Poly{2} - modxk(k) * x)).modxk(k);
        }
        return x.modxk(m);
    }
    Poly log(int m) const {
        return (deriv() * inv(m)).integr().modxk(m);
    }
    Poly exp(int m) const {
        Poly x{1};
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x * (Poly{1} - x.log(k) + modxk(k))).modxk(k);
        }
        return x.modxk(m);
    }
    Poly pow(int k, int m) const {
        int i = 0;
        while (i < size() && a[i].val() == 0) {
            i++;
        }
        if (i == size() || 1LL * i * k >= m) {
            return Poly(std::vector<Z>(m));
        }
        Z v = a[i];
        auto f = divxk(i) * v.inv();
        return (f.log(m - i * k) * k).exp(m - i * k).mulxk(i * k) * power(v, k);
    }
    Poly sqrt(int m) const {
        Poly x{1};
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x + (modxk(k) * x.inv(k)).modxk(k)) * ((P + 1) / 2);
        }
        return x.modxk(m);
    }
    Poly mulT(Poly b) const {
        if (b.size() == 0) {
```

```
            return Poly();
        }
        int n = b.size();
        std::reverse(b.a.begin(), b.a.end());
        return ((*this) * b).divxk(n - 1);
    }
    std::vector<Z> eval(std::vector<Z> x) const {
        if (size() == 0) {
            return std::vector<Z>(x.size(), 0);
        }
        const int n = std::max(int(x.size()), size());
        std::vector<Poly> q(4 * n);
        std::vector<Z> ans(x.size());
        x.resize(n);
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                q[p] = Poly{1, -x[l]};
            } else {
                int m = (l + r) / 2;
                build(2 * p, l, m);
                build(2 * p + 1, m, r);
                q[p] = q[2 * p] * q[2 * p + 1];
            }
        };
        build(1, 0, n);
        std::function<void(int, int, int, const Poly &)> work = [&](int p, int l, int
r, const Poly &num) {
            if (r - l == 1) {
                if (l < int(ans.size())) {
                    ans[l] = num[0];
                }
            } else {
                int m = (l + r) / 2;
                work(2 * p, l, m, num.mulT(q[2 * p + 1]).modxk(m - l));
                work(2 * p + 1, m, r, num.mulT(q[2 * p]).modxk(r - m));
            }
        };
        work(1, 0, n, mulT(q[1].inv(n)));
        return ans;
    }
};
```

取模类(int)

```
template<class T>
constexpr T power(T a, i64 b) {
    T res = 1;
    while(b)
    {
        if(b & 1)res *= a;
        a *= a;
        b >>= 1;
```

```
    }
    return res;
}

template<int P>
struct MInt {
    int x;
    constexpr MInt() : x{} {}
    constexpr MInt(i64 x) : x{norm(x % getMod())} {}

    static int Mod;
    constexpr static int getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(int Mod_) {
        Mod = Mod_;
    }
    constexpr int norm(int x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr int val() const {
        return x;
    }
    explicit constexpr operator int() const {
        return x;
    }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
        return res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return power(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
        x = 1LL * x * rhs.x % getMod();
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
```

```
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & {
        return *this *= rhs.inv();
    }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr std::istream &operator>>(std::istream &is, MInt &a) {
        i64 v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr std::ostream &operator<<(std::ostream &os, const MInt &a) {
        return os << a.val();
    }
    friend constexpr bool operator==(MInt lhs, MInt rhs) {
        return lhs.val() == rhs.val();
    }
    friend constexpr bool operator!=(MInt lhs, MInt rhs) {
        return lhs.val() != rhs.val();
    }
};

template<>
int MInt<0>::Mod = 998244353;

//常数逆元
template<int V, int P>
constexpr MInt<P> CInv = MInt<P>(V).inv();

constexpr int P = 1000000007;//使用模数
using Z = MInt<P>;
```

重心
直径
$Dijkstra$(单源最短路径)
$SPFA$(用于负环判断，差分约束)
Prim(最小生成树)
$Kurskal$
Kruskal重构树
缩点SCC
2-SAT
边双连通分量E-DCC
点双连通分量V-DCC
割点
割边
二分图判断(判奇环)
最大二分图匹配
最大流
最小费用最大流
欧拉回路/路径
点分治
树上启发式合并

图论

重心

```
void solve()
{
    int n;
    std::cin >> n;
    std::vector<std::vector<int>>g(n + 1);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::vector<int>siz(n + 1),fl(n + 1),cg;
    auto dfs = [&](auto &&self,int u,int f) -> void
    {
        siz[u] = 1;
        int mx = 0;
        for(int v : g[u])
        {
            if(v == f)continue;
            self(self,v,u);
            mx = std::max(mx,siz[v]);
            siz[u] += siz[v];
        }
```

```
        mx = std::max(mx,n - siz[u]);
        fl[u] = mx <= (n >> 1);
        if(fl[u])cg.push_back(u);
    };
    dfs(dfs,1,0);
}
```

直径

```
void solve()
{
    int n;
    std::cin >> n;
    std::vector<std::vector<int>>g(n + 1);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::vector<int>dep(n + 1);
    int d1,d2;
    auto dfs = [&](auto &&self,int u,int f,int &mx) -> void
    {
        if(dep[u] > dep[mx])mx = u;
        for(int v : g[u])
        {
            if(v == f)continue;
            dep[v] = dep[u] + 1;
            self(self,v,u,mx);
        }
    };
    d1 = 1,dep[1] = 0;
    dfs(dfs,1,0,d1);
    d2 = d1,dep[d1] = 0;
    dfs(dfs,d1,0,d2);
}
```

Dijkstra(单源最短路径)

```
void dijkstra(int st)
{
    int n;
    vector<int>dis(n + 1,INF);
    vector<bool>vis(n + 1);
    dis[st] = 0;
    priority_queue<PA>q;
    q.push({-dis[st],st});
```

```
    while(!q.empty())
    {
        auto [d,u] = q.top();q.pop();
        if(vis[u])continue;
        vis[u] = 1;
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(vis[v])continue;
            if(dis[v] > dis[u] + e[i].w)
            {
                dis[v] = dis[u] + e[i].w;
                q.push({dis[v],v});
            }
        }
    }
}
```

$SPFA$(用于负环判断，差分约束)

```
bool SPFA()
{
    while(!q.empty())
    {
        int u = q.front();
        q.pop();
        vis[u] = 0;
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(dis[u] + e[i].w < dis[v])
            {
                dis[v] = dis[u] + e[i].w;
                if(!vis[v])
                {
                    q.push(v);
                    vis[v] = 1;
                    //计算最短路时可不需要下面
                    cnt[v]++;
                    if(cnt[v]>n)return true;
                }
            }
        }
    }
    return false;
}
//差分约束建边
求解一个变量序列最大值或最小值，可通过建立对应差分约束系统
求解最短路径或者最长路径来实现,其最短路或最长路就是这个变量可能的最大值或最小值
最短路径i + k >= j,add(i,j,k);
最长路径i + k <= j,add(i,j,k);
```

Prim(最小生成树)

```
void prim()
{
    int tot = 0,ans = 0;
    priority_queue<PA>q;
    vector<int>dis(n + 1,INF);
    vector<bool>vis(n + 1);
    dis[1] = 0;
    q.push({-dis[1],1});
    while(!q.empty())
    {
        auto [d,u] = q.top();q.pop();
        d = -d;
        if(d != dis[u])continue;
        vis[u] = 1;
        ans += dis[u];
        tot++;
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(!vis[v] && dis[v] > e[i].w)
            {
                dis[v] = e[i].w;
                q.push({v,dis[v]});
            }
        }
    }
    if(tot != n)ans = -1;//无法生成一颗树
}
```

Kurskal

```
void kruskal()
{
    sort(edge,edge + m,cmp);
    for(re int i = 0;i < m;i++)
    {
        eu = find(edge[i].u), ev = find(edge[i].v);
        if(eu == ev)
        {
            continue;
        }
        ans += edge[i].w;
        fa[ev] = eu;
        if(++cnt == n - 1)
        {
            break;
        }
    }
}
```

Kruskal重构树

```
std::vector<std::array<int,3>>edge(m);
std::sort(edge.begin(),edge.end());
std::vector g(2 * n ,std::vector<int>());
std::vector<int>w(2 * n);
dsu ds = dsu(2 * n);//并查集
int tot = n;
for(auto [w,u,v] : edge)
{
    int pu = ds.find(u);
    int pv = ds.find(v);
    if(pu == pv)continue;
    ds.fa[pu] = ++tot;
    ds.fa[pv] = tot;
    w[tot] = w;
    g[tot].push_back(pu);
    g[tot].push_back(pv);
}
```

缩点SCC

```
int low[N],dfn[N],times = 0;
int belong[N],tot,dis[N];//缩点后每个点的归属，缩点后数量,缩点的权值
bool vis[N];//当前点是否正在被缩
stack<int>st;
void tarjan(int p)
{
    low[p] = dfn[p] = ++times;
    vis[p] = 1;
    st.push(p);
    for (int i = heads[p];i; i = e[i].nx)
    {
        int v = e[i].to;
        if (!dfn[v])
        {
            tarjan(v);
            low[p] = min(low[p], low[v]);
        }
        else if (vis[v])
            low[p] = min(low[p], dfn[v]);
    }
    if (low[p] == dfn[p])
    {
        tot++;
        while (1)
        {
            int x = st.top();
            st.pop();
            belong[x] = tot;
```

```
            vis[x] = 0;
            if (x == p)break;
        }
    }
}
```

2-SAT

```
int n,m;
std::cin >> n >> m;
std::vector g(n * 2 + 1,std::vector<int>());
for(int i = 1;i <= m;i++)
{
    int u,op1,v,op2;
    std::cin >> u >> op1 >> v >> op2;
    g[!op1 * n + u].push_back(op2 * n + v);
    g[!op2 * n + v].push_back(op1 * n + u);
}
std::vector<int>dfn(2 * n + 1),low(2 * n + 1),vis(2 * n + 1),bel(2 * n + 1);
std::stack<int>stk;
int times = 0,tot = 0;
auto dfs = [&](auto &&self,int u) -> void
{
    vis[u] = 1;
    stk.push(u);
    dfn[u] = low[u] = ++times;
    for(int v : g[u])
    {
        if(!dfn[v])
        {
            self(self,v);
            low[u] = std::min(low[u],low[v]);
        }
        else if(vis[v])low[u] = std::min(low[u],dfn[v]);
    }
    if(dfn[u] == low[u])
    {
        tot++;
        while(!stk.empty())
        {
            int x = stk.top();stk.pop();
            bel[x] = tot;
            vis[x] = 0;
            if(x == u)break;
        }
    }
};
for(int i = 1;i <= 2 * n;i++)
    if(!dfn[i])dfs(dfs,i);

for(int i = 1;i <= n;i++)
    if(bel[i] == bel[i + n])
```

```
    {
        std::cout << "IMPOSSIBLE\n";
        return;
    }
std::cout << "POSSIBLE\n";
for(int i = 1;i <= n;i++)
    std::cout << (bel[i + n] < bel[i]) << " \n"[i == n];
```

边双连通分量E-DCC

```
struct edge
{
    int nx;
    int to;
}e[M];int heads[N],cntqq = 1;
void add(int u,int v)
{
    e[++cntqq] = {heads[u],v};
    heads[u] = cntqq;
}
void solve()
{
    //初始化
    cntqq = 1;
    memset(heads,0,sizeof(int) * (n + 1));

    std::vector<int>low(n + 1),dfn(n + 1),bel(n + 1);//属于哪个缩点
    std::vector<bool>bz(2 * (m + 1));//桥
    std::vector<std::vector<int>>e_dcc(n + 1);//缩点的集合
    int tot = 0,times = 0;
    auto tarjan = [&](auto &&self,int u,int p) -> void
    {
        low[u] = dfn[u] = ++times;
        for (int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if (!dfn[v])
            {
                self(self,v,i);
                if(low[v] > dfn[u])bz[i] = bz[i ^ 1] = 1;
                low[u] = std::min(low[u], low[v]);
            }
            else if(i != (p ^ 1))
                low[u] = std::min(low[u], dfn[v]);
        }
    };
    auto dfs1 = [&](auto &&self,int u,int id) -> void
    {
        bel[u] = id;
        e_dcc[id].push_back(u);
```

```
        for(int i = heads[u];i;i = e[i].nx)
        {
            int v = e[i].to;
            if(bel[v] || bz[i])continue;
            self(self,v,id);
        }
    };
    for(int i = 1;i <= n;i++)
        if(!dfn[i])tarjan(tarjan,i,0);
    for(int i = 1;i <= n;i++)
        if(!bel[i])dfs1(dfs1,i,++tot);
}
```

点双连通分量V-DCC

```
std::vector<int>v_dcc[N];
int tot,times;
void solve()
{
    tot = times = 0;
    int n,m;
    std::cin >> n >> m;
    std::vector<std::vector<int>>g(n + 1);
    std::vector<int>low(n + 1),dfn(n + 1),cut(n + 1);
    for(int i = 1;i <= m;i++)
    {
        int u,v;
        std::cin >> u >> v;
        if(u == v)continue;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::stack<int>stk;
    auto tarjan = [&](auto &&self,int u,int rt) -> void
    {
        low[u] = dfn[u] = ++times;
        stk.push(u);
        if(g[u].empty())//孤立点
        {
            v_dcc[++tot].push_back(u);
            return;
        }
        int cnt = 0;//子树数量
        for(int v : g[u])
        {
            if(!dfn[v])
            {
                self(self,v,rt);
                low[u] = std::min(low[u],low[v]);
                if(low[v] >= dfn[u])//作为割点
                {
                    cnt++,tot++;
```

```
                    while(1)
                    {
                        int x = stk.top();stk.pop();
                        v_dcc[tot].push_back(x);
                        if(x == v)break;
                    }
                    v_dcc[tot].push_back(u);
                }
            }
            else low[u] = std::min(low[u],dfn[v]);
        }
        cut[u] = cnt > (u == rt);
    };
    for(int i = 1;i <= n;i++)
        if(!dfn[i])tarjan(tarjan,i,i);
    std::cout << tot << std::endl;
    for(int i = 1;i <= tot;i++)
    {
        int len = v_dcc[i].size();
        std::cout << len << " ";
        for(int j = 0;j < len;j++)
            std::cout << v_dcc[i][j] << " \n"[j == len - 1];
    }
}
```

割点

```
std::vector<int>cut(n + 1),low(n + 1),dfn(n + 1);
int times = 0;
auto tarjan = [&](auto &&self,int u,int rt) -> void
{
    dfn[u] = low[u] = ++times;
    int cnt = 0;//子树数量
    for(int i = heads[u];i;i = e[i].nx)
    {
        int v = e[i].to;
        if(!dfn[v])
        {
            self(self,v,rt);
            low[u] = std::min(low[u],low[v]);
            if(low[v] >= dfn[u])cnt++;
        }
        else
            low[u] = std::min(low[u],dfn[v]);
    }
    cut[u] = cnt > (rt == u);
};
```

割边

```
//初始化
```

```
cntqq = 1;

std::vector<bool>bz(2 * (m + 1));//从编号 2 开始
std::vector<int>low(n + 1),dfn(n + 1);
int times = 0;
//p代表到达u的边编号
auto tarjan = [&](auto &&self,int u,int p) -> void
{
    low[u] = dfn[u] = ++times;
    for (int i = heads[u];i;i = e[i].nx)
    {
        int v = e[i].to;
        if (!dfn[v])
        {
            self(self,v,i);
            if(low[v] > dfn[u])bz[i] = bz[i ^ 1] = 1;
            low[u] = std::min(low[u], low[v]);
        }
        else if(i != (p ^ 1))//不走反向边
            low[u] = std::min(low[u], dfn[v]);
    }
};
```

二分图判断(判奇环)

```
vector<bool>vis(n + 1);
vector<int>col(n + 1);
auto dfs = [&](auto &&self,int u) -> bool
{
    vis[u] = true;
    for(int i = heads[u];i;i = e[i].nx)
    {
        int v = e[i].to;
        if(vis[v])
        {
            if(col[u] == col[v])return false;
        }
        else
        {
            col[v] = col[u] ^ 1;
            if(!self(self,v))return false;
        }
    }
    return true;
};
auto check = [&]() -> bool
{
    for(int i = 1;i <= n;i++)
        if(!vis[i])
        {
            col[i] = 0;
            if(!dfs(dfs,i))return false;//是奇环(不是二分图)
```

```
        }
    return true;
}
```

最大二分图匹配

```
//匈牙利算法
bool vis[N][N],ask[N];
int matched[N];
int n, m, e,cnt = 0;
inline bool found(int x)
{
    for (int i = 1; i <= m; i++)
    {
        if (vis[x][i])
        {
            if (ask[i])continue;
            ask[i] = 1;
            if (!matched[i] || found(matched[i]))
            {
                matched[i] = x;
                return true;
            }
        }
    }
    return false;
}

void match()
{
    for (int i = 1; i <= n; i++)
    {
        memset(ask, 0, sizeof ask);
        if (found(i))
            cnt++;
    }
}
void solve()
{
    cin >> n >> m >> e;
    for (int i = 1; i <= e; i++)
    {
        int u, v;
        cin >> u >> v;
        vis[u][v] = 1;
    }
    match();
    cout << cnt << endl;
}
signed main() {
```

```
    IOS;
    int T = 1; //cin >> T;
    while (T--) {
        solve();
    }
}
//dinic算法(单位容量下，nsqrt的复杂度，优于匈牙利)
源点指向左部连边
右部指向汇点连边
左部指向右部连边
跑出来的最大流也就是最大匹配
```

最大流

```
//普通 n * nm,单位容量 m * sqrt(n)
struct Dinic
{
    struct edge
    {
        int nx;
        int to;
        int cap;
        int flow;
    };
    int n,cntqq,s,t;
    std::vector<edge>e;
    std::vector<int>heads,cur,dep;
    Dinic(int n):n(n),cntqq(0)
    {
        heads.assign(n + 1,-1);
        cur.resize(n + 1);
        dep.resize(n + 1);
    }
    void add(int u,int v,int cap)
    {
        e.push_back({heads[u],v,cap,0});
        heads[u] = cntqq++;
        e.push_back({heads[v],u,0,0});
        heads[v] = cntqq++;
    };
    bool bfs()
    {
        std::fill(dep.begin(),dep.end(),0);
        std::queue<int>que;
        que.push(s);
        dep[s] = 1;
        while(!que.empty())
        {
            int u = que.front();que.pop();
            for(int i = heads[u];~i;i = e[i].nx)
```

```
            {
                int v = e[i].to;
                if(dep[v] || e[i].cap == e[i].flow)continue;
                dep[v] = dep[u] + 1;
                que.push(v);
            }
        }
        return dep[t];
    }
    int dfs(int u,int flow)
    {
        if(u == t)return flow;
        int res = 0;
        for(int &i = cur[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(e[i].cap <= e[i].flow)continue;
            if(dep[v] != dep[u] + 1)continue;

            int upflow = std::min(e[i].cap - e[i].flow,flow - res);
            int tf = dfs(v,upflow);
            e[i].flow += tf;
            e[i ^ 1].flow -= tf;
            res += tf;
            if(res == flow)break;
        }
        return res;
    }
    int mxflow(int S,int T)
    {
        s = S,t = T;
        int ans = 0;
        while(bfs())
        {
            cur = heads;
            ans += dfs(S,INF);
        }
        return ans;
    }
};
void solve()
{
    int n,m,s,t;
    std::cin >> n >> m >> s >> t;
    Dinic dinic(n);
    for(int i = 1;i <= m;i++)
    {
        int u,v,w;
        std::cin >> u >> v >> w;
        dinic.add(u,v,w);
    }
    std::cout << dinic.mxflow(s,t) << "\n";
}
```

最小费用最大流

```cpp
//无初始负环处理，无势能优化,O(Fnm)
struct Dinic
{
    struct edge
    {
        int nx;
        int to;
        int cap;
        int flow;
        int co;
    };
    int n,cntqq,s,t;
    std::vector<edge>e;
    std::vector<int>heads,cur,dis,dep;
    Dinic(int n):n(n),cntqq(0)
    {
        heads.assign(n + 1,-1);
        cur.resize(n + 1);
        dis.resize(n + 1);
        dep.resize(n + 1);
    }
    void add(int u,int v,int cap,int co)
    {
        e.push_back({heads[u],v,cap,0,co});
        heads[u] = cntqq++;
        e.push_back({heads[v],u,0,0,-co});
        heads[v] = cntqq++;
    };
    bool spfa()
    {
        std::fill(dis.begin(),dis.end(),INF);
        std::fill(dep.begin(),dep.end(),INF);
        std::vector<bool>vis(n + 1);
        dis[s] = dep[s] = 0;
        vis[s] = 1;
        std::queue<int>q;
        q.push(s);
        while(!q.empty())
        {
            int u = q.front();q.pop();
            vis[u] = 0;
            for(int i = heads[u];~i;i = e[i].nx)
            {
                if(e[i].cap <= e[i].flow)continue;
                int v = e[i].to;
                int d = dis[u] + e[i].co;
                if(dis[v] > d || (dis[v] == d && dep[v] > dep[u] + 1))
                {
                    dis[v] = d;
                    dep[v] = dep[u] + 1;
```

```
                    if(!vis[v])
                    {
                        q.push(v);
                        vis[v] = 1;
                    }
                }
            }
        }
        return dis[t] != INF;
    }
    pii dfs(int u,int flow)
    {
        if(u == t)return {flow,0};
        pii res{0,0};
        for(int &i = cur[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(e[i].cap <= e[i].flow)continue;
            if(dis[v] != dis[u] + e[i].co)continue;
            if(dep[v] != dep[u] + 1)continue;

            int upflow = std::min(e[i].cap - e[i].flow,flow - res[0]);
            auto [tf,tc] = dfs(v,upflow);
            e[i].flow += tf;
            e[i ^ 1].flow -= tf;
            res[0] += tf;
            res[1] += tc + tf * e[i].co;
            if(res[0] == flow)break;
        }
        return res;
    }
    pii mxflow(int S,int T,int li = INF)
    {
        s = S,t = T;
        if(s == t)return {0,0};
        pii ans{0,0};
        while(ans[0] < li && spfa())
        {
            cur = heads;
            auto [f,c] = dfs(s,li - ans[0]);
            ans[0] += f;
            ans[1] += c;
        }
        return ans;
    }
};
void solve()
{
    int n,m,s,t;
    std::cin >> n >> m >> s >> t;
    Dinic dinic(n);
    for(int i = 1;i <= m;i++)
    {
```

```
        int u,v,w,c;
        std::cin >> u >> v >> w >> c;
        dinic.add(u,v,w,c);
    }
    auto [f,c] = dinic.mxflow(s,t);
    std::cout << f << " " << c << "\n";
}
```

```
//负环处理,势能优化 + Dijkstra
// O(Knm + F(n + m)log(m))
//K为消环次数，F为最大流量
struct Dinic
{
    struct edge
    {
        int nx;
        int to;
        int cap;
        int flow;
        int co;
    };
    int cntqq,n,s,t;
    std::vector<edge>e;
    std::vector<int>heads,cur,dis,dep,h;
    Dinic(int n):cntqq(0),n(n)
    {
        heads.assign(n + 1,-1);
        cur.resize(n + 1);
        dis.resize(n + 1);
        dep.resize(n + 1);
        h.assign(n + 1,0);
    }
    void add(int u,int v,int cap,int co)
    {
        e.push_back({heads[u],v,cap,0,co});
        heads[u] = cntqq++;
        e.push_back({heads[v],u,0,0,-co});
        heads[v] = cntqq++;
    }
    int init()
    {
        int res = 0;
        std::vector<int>pre(n + 1);
        while(1)
        {
            std::fill(h.begin(),h.end(),0);
            std::fill(pre.begin(),pre.end(),-1);
            int x = -1;
            for(int i = 1;i <= n;i++)
            {
                x = -1;
                for(int j = 0;j < cntqq;j++)
```

```
                {
                    if(e[j].cap <= e[j].flow)continue;
                    int u = e[j ^ 1].to,v = e[j].to;
                    if(h[v] > h[u] + e[j].co)
                    {
                        h[v] = h[u] + e[j].co;
                        pre[v] = j;
                        x = v;
                    }
                }
                if(x == -1)break;
            }
            if(x == -1)break;

            for(int i = 1;i <= n;i++)
                x = e[pre[x] ^ 1].to;

            int v = x,f = INF;
            do
            {
                int id = pre[v];
                f = std::min(f,e[id].cap - e[id].flow);
                v = e[id ^ 1].to;
            }while(v != x);
            v = x;
            do
            {
                int id = pre[v];
                e[id].flow += f;
                e[id ^ 1].flow -= f;
                res += f * e[id].co;
                v = e[id ^ 1].to;
            }while(v != x);
        }
        return res;
    }
    bool dij()
    {
        std::fill(dis.begin(),dis.end(),INF);
        std::fill(dep.begin(),dep.end(),INF);
        std::priority_queue<piii,std::vector<piii>,std::greater<piii>>q;
        dis[s] = dep[s] = 0;
        q.push({0,0,s});
        while(!q.empty())
        {
            auto [d,k,u] = q.top();q.pop();
            if(d != dis[u] || k != dep[u])continue;
            for(int i = heads[u];~i;i = e[i].nx)
            {
                if(e[i].cap <= e[i].flow)continue;
                int v = e[i].to;
                int nd = d + e[i].co + h[u] - h[v];
                if(dis[v] > nd || (dis[v] == nd && dep[v] > k + 1))
```

```
                {
                    dis[v] = nd;
                    dep[v] = k + 1;
                    q.push({dis[v],dep[v],v});
                }
            }
        }
        if(dis[t] == INF)return false;
        for(int i = 1;i <= n;i++)
            if(dis[i] != INF)h[i] += dis[i];
        return true;
    }
    pii dfs(int u,int flow)
    {
        if(u == t)return {flow,0};
        pii res{0,0};
        for(int &i = cur[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            if(e[i].cap <= e[i].flow)continue;
            if(h[v] != h[u] + e[i].co)continue;
            if(dep[v] != dep[u] + 1)continue;

            int upflow = std::min(e[i].cap - e[i].flow,flow - res[0]);
            auto [tf,tc] = dfs(v,upflow);
            e[i].flow += tf;
            e[i ^ 1].flow -= tf;
            res[0] += tf;
            res[1] += tc + tf * e[i].co;
            if(res[0] == flow)break;
        }
        return res;
    }
    pii mxflow(int S,int T,int li = INF)
    {
        s = S,t = T;
        if(s == t)return {0,0};
        pii ans{0,init()};
        while(ans[0] < li && dij())
        {
            cur = heads;
            auto [f,c] = dfs(s,li - ans[0]);
            ans[0] += f;
            ans[1] += c;
        }
        return ans;
    }
};
void solve()
{
    int n,m,s,t;
    std::cin >> n >> m >> s >> t;
    Dinic dinic(n);
```

```
    for(int i = 1;i <= m;i++)
    {
        int u,v,w,c;
        std::cin >> u >> v >> w >> c;
        dinic.add(u,v,w,c);
    }
    auto [f,c] = dinic.mxflow(s,t);
    std::cout << f << " " << c << "\n";
}
```

欧拉回路/路径

```
//记得判连通
//两种都是对于字典序最小的欧拉回路/路径的输出
//对于无向图
int n,m;
cin >> n >> m;
vector<vector<PA>>g(n + 1);
vector<int>du(n + 1),heads(n + 1);
vector<bool>vis(m + 1);
for(int i = 1;i <= m;i++)
{
    int u,v;
    cin >> u >> v;
    g[u].push_back({v,i});
    g[v].push_back({u,i});
    du[u]++;
    du[v]++;
}
int st = 0,ed = 0;
for(int i = 1;i <= n;i++)
{
    if(du[i] & 1)
    {
        if(!st)st = i;
        else if(!ed)ed = i;
        else
        {
            cout << "No" << endl;
            return;
        }
    }
}
st = st ? st : 1;
for(int i = 1;i <= n;i++)
    sort(g[i].begin(),g[i].end());
stack<int>stk;
auto dfs = [&](auto &&self,int x) -> void
{
    for(int &i = heads[x];i < g[x].size();)
    {
        auto [v,op] = g[x][i++];
```

```
        if(vis[op])continue;
        vis[op] = 1;
        self(self,v);
    }
    stk.push(x);
};
dfs(dfs,st);
while(!stk.empty())
{
    cout << stk.top() << " ";
    stk.pop();
}
cout << endl;
//对于有向图
int n,m;
cin >> n >> m;
vector<vector<int>>g(n + 1);
vector du(n + 1,vector<int>(2));
vector<int>heads(n + 1);
for(int i = 1;i <= m;i++)
{
    int u,v;
    cin >> u >> v;
    g[u].push_back(v);
    du[v][0]++;
    du[u][1]++;
}
int st = 0,ed = 0;
bool op = 0;
for(int i = 1;i <= n;i++)
{
    if(du[i][0] == du[i][1])continue;
    else if(du[i][1] - du[i][0] == 1)
    {
        if(!st)st = i;
        else op = 1;
    }
    else if(du[i][0] - du[i][1] == 1)
    {
        if(!ed)ed = i;
        else op = 1;
    }
    else op = 1;
}
if(op)
{
    cout << "No" << endl;
    return;
}
for(int i = 1;i <= n;i++)
    sort(g[i].begin(),g[i].end());
st = st ? st : 1;
stack<int>stk;
```

```
auto dfs = [&](auto && self,int x) -> void
{
    for(int &i = heads[x];i < g[x].size();)
    {
        int v = g[x][i++];
        self(self,v);
    }
    stk.push(x);
};
dfs(dfs,st);
while(!stk.empty())
{
    cout << stk.top() << " ";
    stk.pop();
}
cout << endl;
```

点分治

```
//求询问的点对距离是否存在，点分治本体nlogn

std::vector<int>siz(n + 1),vis(n + 1),mx(n + 1);//找重心 + 删点
//连通块大小
auto get_siz = [&](auto &&self,int u,int f) -> int
{
    int sum = 1;
    for(auto [v,w] : g[u])
    {
        if(vis[v] || v == f)continue;
        sum += self(self,v,u);
    }
    return sum;
};
//重心为根
auto get_rt = [&](auto &&self,int u,int f,int sum,int &rt) -> void
{
    mx[u] = 0;
    siz[u] = 1;
    for(auto [v,w] : g[u])
    {
        if(vis[v] || v == f)continue;
        self(self,v,u,sum,rt);
        mx[u] = std::max(mx[u],siz[v]);
        siz[u] += siz[v];
    }
    mx[u] = std::max(mx[u],sum - siz[u]);
    if(mx[u] < mx[rt])rt = u;
};
//根据题目具体情况来写
auto calc = [&](int u) -> void
{
    std::vector<int>S(1,0);
```

```
    exist[0] = 1;
    auto get_dis = [&](auto &&self,int now,int f,int dis,std::vector<int>&tp) -> void
    {
        if(dis >= N)return;//小剪枝

        tp.push_back(dis);
        for(int i = 0;i < m;i++)
            if(0 <= que[i] - dis && que[i] - dis < N && exist[que[i] - dis])
                ans[i] = 1;
        for(auto [v,w] : g[now])
        {
            if(v == f || vis[v])continue;
            self(self,v,now,dis + w,tp);
        }
    };
    for(auto [v,w] : g[u])
    {
        if(vis[v])continue;
        std::vector<int>tmp;
        get_dis(get_dis,v,u,w,tmp);
        for(int x : tmp)
        {
            if(x < N)
            {
                S.push_back(x);
                exist[x] = 1;
            }
        }
    }
    for(int x : S)
        if(x < N)exist[x] = 0;
};
auto divide = [&](auto &&self,int u) -> void
{
    int sum = get_siz(get_siz,u,0);
    int rt = 0;mx[rt] = INF;
    get_rt(get_rt,u,0,sum,rt);

    calc(rt);
    vis[rt] = 1;
    for(auto [v,w] : g[rt])
        if(!vis[v])
            self(self,v);
};
divide(divide,1);


```

树上启发式合并

```
void solve()
{
```

```cpp
    int n;
    std::cin >> n;
    std::vector<std::vector<int>>g(n + 1);
    for(int i = 1;i < n;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    std::vector<int>col(n + 1);
    for(int i = 1;i <= n;i++)std::cin >> col[i];

    std::vector<int>siz(n + 1),son(n + 1),dfn(n + 1),ind(n + 1);
    int tot = 0;
    auto dfs1 = [&](auto &&self,int u,int fa) -> void
    {
        dfn[u] = ++tot;
        ind[tot] = u;
        siz[u] = 1;
        int zson = -1;
        for(int v : g[u])
        {
            if(v == fa)continue;
            self(self,v,u);
            siz[u] += siz[v];
            if(siz[v] > zson)
            {
                son[u] = v;
                zson = siz[v];
            }
        }
    };
    dfs1(dfs1,1,0);
    std::vector<int>cnt(n + 1),ans(n + 1);
    int sum = 0;
    auto dfs = [&](auto &&self,int u,int fa,bool fl) -> void
    {
        for(int v : g[u])
        {
            if(v == fa || v == son[u])continue;
            self(self,v,u,0);
        }
        if(son[u])self(self,son[u],u,1);
        if(!cnt[col[u]])sum++;
        cnt[col[u]]++;
        for(int v : g[u])
        {
            if(v == fa || v == son[u])continue;
            for(int i = dfn[v];i <= dfn[v] + siz[v] - 1;i++)
            {
                int pos = ind[i];
                if(!cnt[col[pos]])sum++;
```

```
                cnt[col[pos]]++;
            }
        }
        ans[u] = sum;
        if(fl)return;
        for(int i = dfn[u];i <= dfn[u] + siz[u] - 1;i++)
        {
            int pos = ind[i];
            cnt[col[pos]]--;
            if(!cnt[col[pos]])sum--;
        }
    };
    dfs(dfs,1,0,1);
    int m;std::cin >> m;
    while(m--)
    {
        int u;std::cin >> u;
        std::cout << ans[u] << "\n";
    }
}
```

Linux下*Makefile*配置
Windows下Makefile配置
Random,随机数生成模板
acm代码模板
快读快写
排序与高精度
    归并排序求逆序对
    高精度加法
    高精度减法
    高精乘高精
异或哈希
普通莫队
单点修改莫队
CDQ分治(三维偏序)

算法板子

Linux下*Makefile*配置

```
# 最简版(需要acm.cpp,in,out,Answer)
CXX := g++
CXXFLAGS := -Wall -std=c++17 -lm -O3 -Ofast -fsanitize=address -fsanitize=undefined -fno-omit-frame-pointer

.DEFAULT_GOAL := run

build/acm: acm.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

# 普通运行
.PHONY: run
run: build/acm
    @build/acm < in
```

```
CXX := g++
CXXFLAGS := -Wall -std=c++17 -lm -O3 -Ofast
TEST ?= 100

.DEFAULT_GOAL := run

build/acm: acm.cpp
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

build/std: std.cpp
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

```

```
build/random: random.cpp
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

.PHONY: run
run: build/acm
    @./build/acm < in

.PHONY: dp
dp: build/acm build/std build/random
    @for i in $$(seq 1 $(TEST)); do \
        ./build/random > in; \
        ./build/acm < in > out; \
        ./build/std < in > stdout; \
        if ! diff -w out stdout > /dev/null; then \
            echo "Wrong Answer on test $$i"; \
            echo "Input:"; cat in; \
            echo "ACM output:"; cat out; \
            echo "STD output:"; cat stdout; \
            exit 1; \
        fi; \
    done
    @echo "Accepted, passed $(TEST) tests"
```

```

# 完整版(还需要std.cpp,random.cpp,stdout)
CXX := g++
CXXFLAGS := -Wall -std=c++17 -lm -O3 -Ofast -fsanitize=address -fsanitize=undefined -fno-omit-frame-pointer

.DEFAULT_GOAL := run

build/acm: acm.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

build/std: std.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

build/random: random.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

# 普通运行
.PHONY: run
run: build/acm
    @build/acm < in

# 测试样例
.PHONY: check
```

```
check: build/acm
    @build/acm < in > out
    @if ! diff -q out Answer > /dev/null; then \
        echo "Wrong Answer"; \
        echo "Input:"; \
        cat in; \
        echo "Output:"; \
        cat out; \
        echo "Answer:"; \
        cat Answer; \
        exit 1; \
    fi
    @echo "Accepted"

# 测试特殊点，与暴力std比较
.PHONY: cp
cp: build/acm build/std
    @build/acm < in > out
    @build/std < in > stdout
    @if ! diff -q out stdout > /dev/null; then \
        echo "Wrong Answer"; \
        echo "Input:"; \
        cat in; \
        echo "ACM output:"; \
        cat out; \
        echo "STD output:"; \
        cat stdout; \
        exit 1; \
    fi
    @echo "Accepted"

# 对拍
.PHONY: dp
dp: build/acm build/std build/random
    @echo "Running test cases..."
    @for i in $$(seq 1 10); do \
        echo "[Test $$i]"; \
        build/random > in; \
        build/acm < in > out; \
        build/std < in > stdout; \
        if ! diff -q out stdout > /dev/null; then \
            echo "Wrong Answer on test $$i"; \
            echo "Input:"; \
            cat in; \
            echo "ACM output:"; \
            cat out; \
            echo "STD output:"; \
            cat stdout; \
            exit 1; \
        fi; \
    done
    @echo "Accepted"
```

Windows下Makefile配置

```
CXX := g++
CXXFLAGS := -Wall -std=c++23 -lm -O3
TARGET_DIR := build

.DEFAULT_GOAL := run

ensure-dir:
    @mkdir -p "$(TARGET_DIR)"

$(TARGET_DIR)/acm.exe: acm.cpp Makefile | ensure-dir
    $(CXX) $< -o $@ $(CXXFLAGS)
# 普通运行
.PHONY: run
run: $(TARGET_DIR)/acm.exe
    @./$(TARGET_DIR)/acm.exe < in
```

Random,随机数生成模板

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

i64 rand_int(i64 l, i64 r) {
    return uniform_int_distribution<i64>(l, r)(rng);
}

void solve() {

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
    return 0;
}
```

acm代码模板

```
#include<bits/stdc++.h>
#define IOS ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
#define int long long
#define inf 0x3f3f3f3f
```

```cpp
#define INF 4557430888798830399
using ull = unsigned long long;
using pii = std::pair<int,int>;
typedef complex<double> cp;
const int N = 1e6 + 7;
const int M = 2e6 + 7;
const int mod = 998244353;
const double pi = acos(-1);
const double e = exp(1);

void solve()
{

}
signed main()
{
    // init();
    IOS;
    int T = 1;cin >> T;
    while(T--)
    {
        solve();
    }
    return 0;
}
```

快读快写

```cpp
inline int read()
{
    int x = 0,f = 1;
    char ch = getchar();
    while(ch < '0' || ch > '9')
    {
        if(ch == '-')
            f = -1;
        ch = getchar();
    }
    while(ch >= '0' && ch <= '9')
        x = x * 10 + ch - '0',ch = getchar();
    return x * f;
}
void print(int x)
{
    if(x < 0)
        putchar('-'),x = -x;
    if(x > 9)
        print(x / 10);
    putchar(x % 10 + '0');
    return;
}
```

排序与高精度

归并排序求逆序对

```
int a[N],c[N];
int cnt;
void msort(int l,int r)
{
    if(l >= r)return;
    int i = l,mid = (l + r) >> 1,j = mid + 1,k = l;
    msort(l,mid),msort(mid + 1,r);
    while(i <= mid && j <= r)
    {
        if(a[i] <= a[j])c[k++] = a[i++];
        else c[k++] = a[j++],cnt += mid - i + 1;
    }
    while(i <= mid)c[k++] = a[i++];
    while(j <= r)c[k++] = a[j++];
    for(int x = l;x <= r;x++)a[x] = c[x];
}
```

高精度加法

```
map<char, int>mp;
void init()
{
    for (int i = 0; i <= 9; i++)mp['0' + i] = i;
    for (int i = 0; i <= 25; i++)
    {
        mp['A' + i] = i + 10;
        mp['a' + i] = i + 36;
    }
}
string add(string a, string b, int z)//z是进制
{
    string ans = "";
    int c = 0;
    for (int i = a.size() - 1, j = b.size() - 1; i >= 0 || j >= 0; i--, j--)
    {
        if (i >= 0 && j >= 0)
        {
            int a1 = mp[a[i]], b1 = mp[b[j]];//映射的是字母代表的数
            int sum = a1 + b1 + c;
            if (sum - z >= 0)
            {
                c = 1;
            }
            else c = 0;
            sum %= z;
            if (sum <= 9)ans = (char)('0' + sum) + ans;
            else if (sum <= 35)ans = (char)('A' + sum - 10) + ans;
```

```
            else ans = (char)('a' + sum - 36) + ans;
        }
        else if (j >= 0)
        {
            int a1 = 0, b1 = mp[b[j]];//映射的是字母代表的数
            int sum = a1 + b1 + c;
            if (sum - z >= 0)
            {
                c = 1;
            }
            else c = 0;
            sum %= z;
            if (sum <= 9)ans = (char)('0' + sum) + ans;
            else if (sum <= 35)ans = (char)('A' + sum - 10) + ans;
            else ans = (char)('a' + sum - 36) + ans;
        }
        else
        {
            int a1 = mp[a[i]], b1 = 0;
            int sum = a1 + b1 + c;
            if (sum - z >= 0)
            {
                c = 1;
            }
            else c = 0;
            sum %= z;
            if (sum <= 9)ans = (char)('0' + sum) + ans;
            else if (sum <= 35)ans = (char)('A' + sum - 10) + ans;
            else ans = (char)('a' + sum - 36) + ans;
        }
    }
    if (c)
        ans = '1' + ans;
    return ans;
}
```

高精度减法

```
bool cmp(vector<int>& A, vector<int>& B)
{
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = A.size() - 1; i >= 0; --i)
        if (A[i] != B[i])
            return A[i] > B[i];

    return true;
}
vector<int> sub(vector<int>& A, vector<int>& B)
{
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size(); ++i)
```

```
    {
        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        if (t >= 0) t = 0;
        else t = 1;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
void jiefa(string a,string b)
{
    vector<int> A, B;
    for (int i = a.size() - 1; i >= 0; --i) A.push_back(a[i] - '0');
    for (int i = b.size() - 1; i >= 0; --i) B.push_back(b[i] - '0');
    vector<int> C;
    if (cmp(A, B)) C = sub(A, B);
    else cout << "-", C = sub(B, A);
    for (int i = C.size() - 1; i >= 0; --i) cout << C[i];
    cout << endl;
}
int main()
{
    string a,b;
    cin >> a >> b;
    jiefa(a,b);
    return 0;
}
```

高精乘高精

```
map<char, int>mp;
void init()
{
    for (int i = 0; i <= 9; i++)mp['0' + i] = i;
    for (int i = 0; i <= 25; i++)
    {
        mp['A' + i] = i + 10;
        mp['a' + i] = i + 36;
    }
}
string cheng(string a_s,string b_s,int z)//z是进制
{
    string ss = "";
    int a[521] = {0},b[521] = {0},ans[521 * 2] = {0};
    int len_a,len_b,len,carry = 0,i,j;
    len_a = a_s.length();
    len_b = b_s.length();

    for(i = 0;i < len_a;i++) a[i] = mp[a_s[len_a - 1 - i]];//映射的是字母代表的数
    for(i = 0;i < len_b;i++) b[i] = mp[b_s[len_b - 1 - i]];//映射的是字母代表的数
    for(i = 0;i < len_b;i++){
```

```
        for(j = 0;j < len_a || carry != 0;j++){
            ans[i + j] += a[j] * b[i] + carry;
            carry = ans[i + j] / z;
            ans[i + j] %= z;
        }
    }
    len = i + j;
    while(ans[len] == 0 && len > 0) len--;
    for(i = len;i >= 0;i--){
        if(ans[i] <= 9)ss += (char)('0' + ans[i]);
        else if(ans[i] <= 35)ss += (char)('A' + ans[i] - 10);
        else ss += (char)('a' + ans[i] - 36);
    }
    return ss;
}
```

异或哈希

```
用哈希降低异或的冲突，通常用来解决一个数出现的次数为奇数还是偶数的情况
const int N = 1e6 + 7;
mt19937_64 rnd(time(0));//
int p[N];//
signed main()
{
    IOS;
    for(int i = 1;i < N;i++)p[i] = rnd();//
    int T = 1;cin >> T;
    while(T--)
    {
        solve();
    }
    return 0;
}

ull p[N];
void init()
{
    p[0] = 1;
    for(int i = 1;i < N;i++)
        p[i] = p[i - 1] * 998244353 + i;
}
```

普通莫队

```
int base;//分块(n / sqrt(m))
struct query
{
    int l;
    int r;
    int id;
    bool operator<(const query &tp)const{
```

```
        if(l / base != tp.l / base)return l < tp.l;
        return (l / base) & 1 ? r < tp.r : r > tp.r;
    };
};
void solve()
{
    int n,m;
    cin >> n >> m;
    base = max(1LL,(int)(n / sqrt(m)));//注意0
    vector<int>a(n + 1);
    vector<query>que(m + 1);
    vector<int>ans1(m + 1),ans2(m + 1),col(n + 1);
    for(int i = 1;i <= n;i++)cin >> a[i];
    for(int i = 1;i <= m;i++)
    {
        int l,r;cin >> l >> r;
        que[i] = {l,r,i};
    }
    sort(que.begin() + 1,que.end());
    int sum = 0;
    //更改
    auto move = [&](int x,int sign) -> void
    {
        if(sign == -1)col[x]--;
        sum += col[x] * sign;
        if(sign == 1)col[x]++;
    };
    for(int i = 1,l = 1,r = 0;i <= m;i++)
    {
        if(que[i].l == que[i].r)
        {
            ans1[que[i].id] = 0;
            ans2[que[i].id] = 1;
            continue;
        }

        while(l > que[i].l)move(a[--l],1);
        while(r < que[i].r)move(a[++r],1);
        while(l < que[i].l)move(a[l++],-1);
        while(r > que[i].r)move(a[r--],-1);
        ans1[que[i].id] = sum;
        ans2[que[i].id] = (r - l + 1) * (r - l) / 2;
    }
    for(int i = 1;i <= m;i++)
    {
        if(!ans1[i])ans2[i] = 1;
        else
        {
            int g = gcd(ans1[i],ans2[i]);
            ans1[i] /= g;
            ans2[i] /= g;
        }
        cout << ans1[i] << "/" << ans2[i] << endl;
```

```
    }
}
```

单点修改莫队

```
int base;//分块
struct query
{
    int l;
    int r;
    int t;
    int id;
    bool operator<(const query& tp)const{
        if(l / base != tp.l / base)return l < tp.l;
        if(r / base != tp.r / base)return r < tp.r;
        return t < tp.t;
    };
};
int cnt[N];
void solve()
{
    int n,m;
    cin >> n >> m;
    base = max(1LL,(int)pow(n,2.00 / 3.00));
    vector<int>a(n + 1);
    for(int i = 1;i <= n;i++)cin >> a[i];
    vector<PA>upt(1);int times = 0;//修改
    vector<query>que(1);int tot = 0;//询问
    for(int i = 1;i <= m;i++)
    {
        char c;cin >> c;
        if(c == 'Q')//询问
        {
            int l,r;
            cin >> l >> r;
            que.push_back({l,r,times,++tot});
        }
        else//修改
        {
            int p,x;
            cin >> p >> x;
            upt.push_back({p,x});
            times++;
        }
    }
    //Mo
    sort(que.begin() + 1,que.end());
    vector<int>ans(tot + 1);
    int sum = 0;
    auto move = [&](int x,int sign)
    {
        if(sign == -1)cnt[x]--;
```

```
        sum += (!cnt[x]) * sign;
        if(sign == 1)cnt[x]++;
    };
    for(int i = 1,l = 1,r = 0,now = 0;i <= tot;i++)
    {
        while(l > que[i].l)move(a[--l],1);
        while(r < que[i].r)move(a[++r],1);
        while(l < que[i].l)move(a[l++],-1);
        while(r > que[i].r)move(a[r--],-1);
        while(now < que[i].t)
        {
            now++;
            if(l <= upt[now].first && upt[now].first <= r)
            {
                move(upt[now].second,1);
                move(a[upt[now].first],-1);
            }
            swap(upt[now].second,a[upt[now].first]);
        }
        while(now > que[i].t)
        {
            if(l <= upt[now].first && upt[now].first <= r)
            {
                move(upt[now].second,1);
                move(a[upt[now].first],-1);
            }
            swap(upt[now].second,a[upt[now].first]);
            now--;
        }
        ans[que[i].id] = sum;
    }
    for(int i = 1;i <= tot;i++)
        cout << ans[i] << endl;
}
```

CDQ分治(三维偏序)

```
struct node
{
    int a;
    int b;
    int c;
    int res;
    int cnt;
    bool operator!=(const node &tp)const{
        if(tp.a != a || tp.b != b || tp.c != c)
            return true;
        return false;
    }
};
bool cmpa(node x,node y)
{
```

```
    if(x.a != y.a)return x.a < y.a;
    if(x.b != y.b)return x.b < y.b;
    return x.c < y.c;
}
bool cmpb(node x,node y)
{
    if(x.b != y.b)return x.b < y.b;
    return x.c < y.c;
}
struct finwick
{
    vector<int>t;
    int n;
    finwick(int len)
    {
        init(len);
    }
    void init(int len)
    {
        n = len;
        t.resize(n + 1);
    }
    int lowbit(int x)
    {
        return x & (-x);
    }
    void add(int x,int k)
    {
        for(int i = x;i <= n;i += lowbit(i))
            t[i] += k;
    }
    int query(int l,int r)
    {
        int ans = 0;
        for(int i = r;i >= 1;i -= lowbit(i))
            ans += t[i];
        for(int i = l - 1;i >= 1;i -= lowbit(i))
            ans -= t[i];
        return ans;
    }
};
void solve()
{
    int n,k;
    cin >> n >> k;
    vector<node>tp(n + 2),arr(1);
    for(int i = 1;i <= n;i++)
    {
        int a,b,c;
        cin >> a >> b >> c;
        tp[i] = {a,b,c,0,0};
    }
    sort(tp.begin() + 1,tp.begin() + 1 + n ,cmpa);
```

```
    int t = 0;
    for(int i = 1;i <= n;i++)
    {
        t++;
        if(tp[i] != tp[i + 1])
        {
            node tmp = tp[i];
            tmp.cnt = t;
            arr.push_back(tmp);
            t = 0;
        }
    }
    int len = arr.size() - 1;
    finwick fi = finwick(2e5);
    auto cdq = [&](auto &&self,int l,int r) -> void
    {
        if(l == r)return;
        int mid = (l + r) >> 1;
        self(self,l,mid);
        self(self,mid + 1,r);
        sort(arr.begin() + l,arr.begin() + 1 + mid,cmpb);
        sort(arr.begin() + 1 + mid,arr.begin() + 1 + r,cmpb);
        int i = l;
        for(int j = mid + 1;j <= r;j++)
        {
            while(i <= mid && arr[i].b <= arr[j].b)
            {
                fi.add(arr[i].c,arr[i].cnt);
                i++;
            }
            arr[j].res += fi.query(1,arr[j].c);
        }
        for(int j = l;j < i;j++)fi.add(arr[j].c,-arr[j].cnt);
    };
    cdq(cdq,1,len);
    vector<int>ans(n);
    for(int i = 1;i <= len;i++)ans[arr[i].res + arr[i].cnt - 1] += arr[i].cnt;
    for(int i = 0;i < n;i++)
        cout << ans[i] << endl;

}
```

字符串哈希
字典树
KMP
KMP自动机
拓展KMP(Z函数)
AC自动机(优化后)
后缀数组
后缀自动机
$Manacher$

字符串

字符串哈希

```
ull p[N];
void init()
{
    p[0] = 1;
    for(int i = 1;i < N;i++)
        p[i] = p[i - 1] * 131;
}
ull get_ha(std::string &s)//获取某一个字符串的哈希值
{
    int n = s.size();
    ull ha = 0;
    for(int i = 1;i <= n;i++)
        ha = ha * 131 + s[i - 1];
    return ha;
}
std::vector<ull>init_hash(std::string &s)//构造s的字符串哈希
{
    int n = s.size();
    std::vector<ull>ha(n + 1);
    for(int i = 1;i <= n;i++)
        ha[i] = ha[i - 1] * 131 + s[i - 1];
    return ha;
}
ull get_hash(std::vector<ull> &ha,int l,int r)//获取子串的哈希值
{
    return ha[r] - ha[l - 1] * p[r - l + 1];
}
```

```
constexpr int P = 1e9 + 7;
//双值哈希
ull p1[N],p2[N];
void init()
{
    p2[0] = p1[0] = 1;
    for(int i = 1;i < N;i++)
```

```
    {
        p1[i] = p1[i - 1] * 131;
        p2[i] = p2[i - 1] * 131 % P;
    }
}
std::vector<ull>init_hash1(std::string &s)
{
    int n = s.size();
    std::vector<ull>ha(n + 1);
    for(int i = 1;i <= n;i++)
        ha[i] = ha[i - 1] * 131 + s[i - 1];
    return ha;
}
std::vector<ull>init_hash2(std::string &s)
{
    int n = s.size();
    std::vector<ull>ha(n + 1);
    for(int i = 1;i <= n;i++)
        ha[i] = ((ha[i - 1] * 131) % P + s[i - 1]) % P;
    return ha;
}
ull get_hash1(std::vector<ull> &ha,int l,int r)
{
    return ha[r] - ha[l - 1] * p1[r - l + 1];
}
ull get_hash2(std::vector<ull> &ha,int l,int r)
{
    return (ha[r] - (ha[l - 1] * p2[r - l + 1]) % P + P) % P;
}
ull get_hash1(std::string &s)
{
    ull ha = 0;
    for(char c : s)
        ha = ha * 131 + c;
    return ha;
}
ull get_hash2(std::string &s)
{
    ull ha = 0;
    for(char c : s)
        ha = (ha * 131 % P + c) % P;
    return ha;
}
```

字典树

```
struct trie
{
    int n,tot;
    std::vector<std::vector<int>>nx;
```

```
    std::vector<int>cnt;
    trie(int n,int m = 26)
    {
        n = n;
        tot = 0;
        nx.resize(n + 1,std::vector<int>(m));
        cnt.resize(n + 1);
    }
    int insert(std::string &s)
    {
        int pos = 0;
        for(char c : s)
        {
            int j = c - 'a';
            if(!nx[pos][j])nx[pos][j] = ++tot;
            pos = nx[pos][j];
        }
        cnt[pos]++;
        return pos;
    }
    bool find(std::string &s)
    {
        int pos = 0;
        for(char c : s)
        {
            int j = c - 'a';
            if(!nx[pos][j])return 0;
            pos = nx[pos][j];
        }
        return cnt[pos];
    }
};
```

KMP

```
std::vector<int> KMP(std::string &s)
{
    int n = s.size();
    std::vector<int>pi(n);
    for(int i = 1;i < n;i++)
    {
        int j = pi[i - 1];
        while(j > 0 && s[i] != s[j])j = pi[j - 1];
        if(s[i] == s[j])j++;
        pi[i] = j;
    }
    return pi;
}
```

```
void KMP(string s,string t)
```

```
{
    int n = s.size();
    int m = t.size();
    vector<int>pi(1 + n);//s + "#"的前缀函数数组
    s = s + '#';
    for(int i = 1;i <= n;i++)
    {
        int j = pi[i - 1];
        while(j > 0 && s[i] != s[j])j = pi[j - 1];
        if(s[i] == s[j])j++;
        pi[i] = j;
    }

    vector<int>ans;//s在t中出现的位置
    int la = 0;
    for(int i = 0;i < m;i++)
    {
        int j = la;
        while(j > 0 && t[i] != s[j])j = pi[j - 1];
        if(t[i] == s[j])j++;
        if(j == n)ans.push_back(i - n + 2);
        la = j;
    }
    for(int v : ans)cout << v << endl;
    for(int i = 0;i < n;i++)
        cout << pi[i] << " ";
    cout << endl;
}


//统计每个前缀在同一个字符串中出现的次数
vector<int> ans(n + 1);
for (int i = 0; i < n; i++) ans[pi[i]]++;
for (int i = n - 1; i > 0; i--) ans[pi[i - 1]] += ans[i];
for (int i = 0; i <= n; i++) ans[i]++;

//统计每个前缀在另一个字符串t中出现的次数
//与上文相似，我们只需要统计t中的前缀函数值，然后叠加即可
vector<int>ans(n + 1);
int la = 0;
for(int i = 0;i < m;i++)
{
    int j = la;
    while(j > 0 && t[i] != s[j])j = pi[j - 1];
    if(t[i] == s[j])j++;
    if(j == n)ans.push_back(i - n + 2);
    la = j;
    ans[la]++;统计
}
for (int i = n - 1; i > 0; i--) ans[pi[i - 1]] += ans[i];叠加
```

KMP自动机

```
void KMP_auto(string s,string t)
{
    int n = s.size();
    int m = t.size();
    std::vector aut(n + 1,std::vector<int>(26));//自动机
    std::vector<int>pi(n + 1);
    s = s + '#';
    for(int i = 0;i <= n;i++)
    {
        for(int c = 0;c < 26;c++)
        {
            if(i > 0 && s[i] - 'A' != c)
                aut[i][c] = aut[pi[i - 1]][c];
            else
                aut[i][c] = i + (s[i] - 'A' == c);
        }
        if(i > 0)pi[i] = aut[pi[i - 1]][s[i] - 'A'];
    }
    std::vector<int>ans;
    int la = 0;
    for(int i = 0;i < m;i++)
    {
        int j = aut[la][t[i] - 'A'];
        if(j == n)ans.push_back(i - n + 2);
        la = j;
    }
    for(int v : ans)
        cout << v << endl;
    for(int i = 0 ;i < n;i++)
        cout << pi[i] << " ";
    cout << endl;
}
```

拓展KMP(Z函数)

```
std::vector<int> Zf(std::string &s)
{
    int n = s.size();
    std::vector<int>z(n);//LCP(s,s[i..n - 1])(最长公共前缀)
    for(int i = 1,l = 0,r = 0;i < n;i++)//[l,r)当前处理区间
    {
        if(i < r)
            z[i] = std::min(r - i,z[i - l]);
        while(i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if(i + z[i] > r)
        {
            l = i;
            r = i + z[i];
```

```
        }
    }
    return z;
}
```

AC自动机(优化后)

```
struct Graph
{
    struct edge
    {
        int nx;
        int to;
        int w;
    };
    std::vector<edge>e;
    std::vector<int>heads;
    Graph(int n = 0,int m = 0) : heads(n + 1,-1)
    {
        e.reserve(m);
    }
    void add(int u,int v,int w = 0)
    {
        e.push_back({heads[u],v,w});
        heads[u] = e.size() - 1;
    }
};
struct AC : Graph
{
    struct node
    {
        std::vector<int>son;
        int fail;
        int cnt;
        int idx;
        void init(int m = 26)
        {
            son.resize(m);
            fail = cnt = idx = 0;
        }
    };
    int tot,pidx;
    std::vector<node>tr;
    AC(int n): Graph(n),tr(n + 1)
    {
        tot = pidx = 0;
        tr[0].init();
    }
    int insert(std::string &s)
    {
        int pos = 0;
        for(char c : s)
```

```
        {
            int j = c - 'a';
            int &son = tr[pos].son[j];
            if(!son)son = ++tot,tr[son].init();
            pos = son;
        }
        if(!tr[pos].idx)tr[pos].idx = ++pidx;
        return tr[pos].idx;
    }
    void build(int m = 26)
    {
        std::queue<int>q;
        for(int i = 0;i < m;i++)
        {
            int son = tr[0].son[i];
            if(son)q.push(son),add(0,son);
        }
        while(!q.empty())
        {
            int u = q.front();q.pop();
            for(int i = 0;i < m;i++)
            {
                if(tr[u].son[i])
                {
                    tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
                    add(tr[tr[u].fail].son[i],tr[u].son[i]);
                    q.push(tr[u].son[i]);
                }
                else tr[u].son[i] = tr[tr[u].fail].son[i];
            }
        }
    }
    void dfs(int u)
    {
        for(int i = heads[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            dfs(v);
            tr[u].cnt += tr[v].cnt;
        }
    }
    std::vector<int> query(std::string &s)
    {
        std::vector<int>ans(pidx + 1);
        int pos = 0;
        for(char c : s)
        {
            int j = c - 'a';
            pos = tr[pos].son[j];
            tr[pos].cnt++;
        }
        dfs(0);
        for(int i = 0;i <= tot;i++)
```

```
            ans[tr[i].idx] += tr[i].cnt;
        return ans;
    }
};
void solve()
{
    int n,sum = 0;std::cin >> n;
    std::vector<std::string>a(n);
    std::vector<int>idx(n);
    for(int i = 0;i < n;i++)
    {
        std::cin >> a[i];
        sum += a[i].size();
    }
    AC ac(sum);
    for(int i = 0;i < n;i++)
        idx[i] = ac.insert(a[i]);
    ac.build();
    std::string s;std::cin >> s;
    auto ans = ac.query(s);
    for(int i = 0;i < n;i++)
        std::cout << ans[idx[i]] << "\n";
}
```

后缀数组

```
lcp(sa[i],sa[j]) = min{ht[i + 1],...ht[j]};
ht[i] = lcp(sa[i],sa[i - 1]);
//lcp(i,j)代表后缀i和后缀j的最长公共后缀
struct SA
{
    string s;
    int n,sa[N],osa[N],rk[N],ork[N],cnt[N],ht[N],st_mn[N][21];
    int lcp(int l,int r)
    {
        l = rk[l],r = rk[r];
        if(l > r)swap(l,r);
        int k = log2(r - l + 1);
        return min(st_mn[l][k],st_mn[r - (1LL << k) + 1][k]);
    }
    void init(string t)
    {
        memset(cnt,0,sizeof(cnt));
        int n = t.size();
        s = " " + t;
        int m = 128,p;
        for(int i = 1;i <= n;i++)cnt[rk[i] = s[i]]++;
        for(int i = 1;i <= m;i++)cnt[i] += cnt[i - 1];
        for(int i = n;i >= 1;i--)sa[cnt[rk[i]]--] = i;
        for(int w = 1;;w <<= 1,m = p)
        {
            int cur = 0;
```

```
            for(int i = n - w + 1;i <= n;i++)osa[++cur] = i;
            for(int i = 1;i <= n;i++)
                if(sa[i] > w)osa[++cur] = sa[i] - w;
            memset(cnt,0,sizeof(int) * (m + 1));
            for(int i = 1;i <= n;i++)cnt[rk[osa[i]]]++;
            for(int i = 1;i <= m;i++)cnt[i] += cnt[i - 1];
            for(int i = n;i >= 1;i--)sa[cnt[rk[osa[i]]]--] = osa[i];
            memcpy(ork,rk,sizeof(int) * (n + 1));
            p = 0;
            for(int i = 1;i <= n;i++)
            {
                if(ork[sa[i]] == ork[sa[i - 1]] && ork[sa[i] + w] == ork[sa[i - 1] +
w])
                    rk[sa[i]] = p;
                else rk[sa[i]] = ++p;
            }
            if(p == n)break;
        }
        for(int i = 1,k = 0;i <= n;i++)
        {
            if(k)k--;
            while(s[sa[rk[i]] + k] == s[sa[rk[i] - 1] + k])k++;
            ht[rk[i]] = k;
        }
        for(int i = 1;i <= n;i++)st_mn[i][0] = ht[i];
        for(int j = 1;j < 21;j++)
            for(int i = 1;i + (1LL << j) - 1 <= n;i++)
                st_mn[i][j] = min(st_mn[i][j - 1],st_mn[i + (1LL << j - 1)][j - 1]);
    }
}sa;
```

后缀自动机

```
/*
    对于u的权值为c的nx边，指向的v，将u的endpos集合 + 1，字符为c的集合传递给v。
    对于根节点，包含所有的endpos
*/
struct SAM
{
    struct state
    {
        int len,link;//当前节点代表的最长的子串，指向的最长后缀
        map<char,int> nx;//指向下一个状态
        void init()
        {
            len = 0;
            link = -1;
            nx.clear();
        }
    }st[N << 1];//节点数量最多2n - 1
    int tot,la;//当前节点数量和当前最长的前缀节点
    void init()
```

```
    {
        tot = la = 0;
        st[0].init();
    }
    void sam_ex(char c)//在线构造后缀自动机
    {
        int cur = ++tot;st[cur].init();
        st[cur].len = st[la].len + 1;//前缀长度为最长子串
        int p = la;
        for(;p != -1 && !st[p].nx.count(c);p = st[p].link)st[p].nx[c] = cur;
        if(p == -1)st[cur].link = 0;
        else
        {
            int q = st[p].nx[c];
            /*
                len(p) + 1是当前字符串最长的一个后缀
                如果这个后缀小于 q 的最长字符串s，说明这个后缀的
                等价位增加了(也就是当前字符串末尾)，且s没有增加
            */
            if(st[p].len + 1 == st[q].len)st[cur].link = q;
            else
            {
                //复制，注意，根据题目维护的其他值不一定相等，需要重新赋值
                int cln = ++tot;st[cln] = st[q];
                //下面的最长子串长度就是例子之一
                st[cln].len = st[p].len + 1;
                for(;p != -1 && st[p].nx.count(c) && st[p].nx[c] == q;p =
st[p].link)st[p].nx[c] = cln;
                st[q].link = st[cur].link = cln;
            }
        }
        la = cur;
    }
}sam;
void solve()
{
    string s;
    cin >> s;
    sam.init();
    for(char c : s)sam.sam_ex(c);
}
```

Manacher

```
//[l,r)区间找中心点，然后计算对称点
std::vector<int> Manacher_odd(std::string &s)
{
    int n = s.size();
    std::vector<int>d1(n,1);//d1[i]: [i - d1[i] + 1,i + d1[i])
    for(int i = 0,l = 0,r = 0;i < n;i++)
    {
        if(i < r)
```

```
            d1[i] = std::min(r - i,d1[l + r - 1 - i]);
        while(i - d1[i] >= 0 && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]])
            d1[i]++;
        if(i + d1[i] > r)
        {
            l = i - d1[i] + 1;
            r = i + d1[i];
        }
    }
    return d1;
}
std::vector<int> Manacher_even(std::string &s)
{
    int n = s.size();

    std::vector<int>d2(n);//d2[i]: [i - d2[i],i + d2[i])
    for(int i = 0,l = 0,r = 0;i < n;i++)
    {
        if(i < r)
            d2[i] = std::min(r - i,d2[l + r - i]);
        while(i - 1 - d2[i] >= 0 && i + d2[i] < n && s[i - 1 - d2[i]] == s[i + d2[i]])
            d2[i]++;
        if(i + d2[i] > r)
        {
            l = i - d2[i];
            r = i + d2[i];
        }
    }
    return d2;
}
```